

SHOWCASE EDITION • same content as the IEEE submission

Bitwise-Reproducible Reduction Across Cluster Reshape

A Compiler/Fabric ABI

Floating-point addition is not associative, so reducing the same bytes on a differently-shaped cluster returns a different answer. This paper fixes one reduction order as an interface contract: the compiler proves it emits that order, and the fabric refuses to execute anything else.

Abstract

Floating-point reductions are not bitwise reproducible under *cluster reshape*: change the worker count or the network and the collective changes its reduction order, and with it the last bits. We measure that premise rather than assert it — reducing identical input *bytes* at fourteen rank counts under PyTorch’s collectives returns fourteen distinct 64-bit roots with spreads reaching 5,120 ulp, while repeats at a fixed shape are bitwise identical. Reduction order is therefore an *interface* concern, and we treat it as a compiler/fabric ABI: a size-indexed contract fixing the operand binding at every internal node of a canonical full binary tree $\text{FBT}(N)$. No conjunct is ours alone; the conjunction is — invariance across network *topologies* rather than resource counts alone, an unmodified IEEE-754 FP64 operator with no wide accumulator, arbitrary non-power-of-two N , and enforcement below the software stack, so that a violated assumption fail-stops instead of returning a quietly different number. We instantiate the ABI as SPRS, a topology-parametric SystemVerilog NoC plus compiler, and state a cross-topology bitwise invariance claim and a presence-gated fail-stop property, connected by a mapping/topology orthogonality proposition. Enumerating the refinement-free face of the compiler’s configuration space exhaustively on the 39 hardware-validated members of a 40-instance suite spanning $N \in [8, 8192]$, $G \in [2, 4096]$ and six direct networks, 2,822 of 2,844 distinct program images settled in RTL simulation and every one returned the canonical root; 22 did not settle, all but one a fabric watchdog fail-stop, and no run returned a finite root differing from it. Injected faults establish oracle non-vacuity on 61–65 of 107 verdicts. The compiler’s search space provably cannot reach the bit result; what it can reach is a fixed portfolio of 4 of 180 configurations that attains the measured optimum on 38 of 39 held-out instances and transfers to a withheld topology, beating every rule we could fit that conditions on the instance. Finally we export the discipline to production collectives in software (*pinned-FBT*), correct the natural index-range construction, and price the repair: bitwise reshape-invariant on all 2,414 cells swept against a negative control that fires, at a geometric mean of $2.57 \times$ stock `ncclAllReduce` — provisional under GPU contention, of which about a factor of two is algorithmic, in bytes moved.

Index terms Deterministic reduction, collective communication, compiler/fabric co-design, IEEE 754 floating point, network-on-chip, bitwise reproducibility, all-reduce.

1 The Problem

Production all-reduce implementations — ring, recursive halving–doubling, double-binary tree — pick the reduction order that maximises throughput for the observed topology and worker count $[?, ?]$. When either changes, so does the order. IEEE 754 addition is non-associative, so the order is part of the

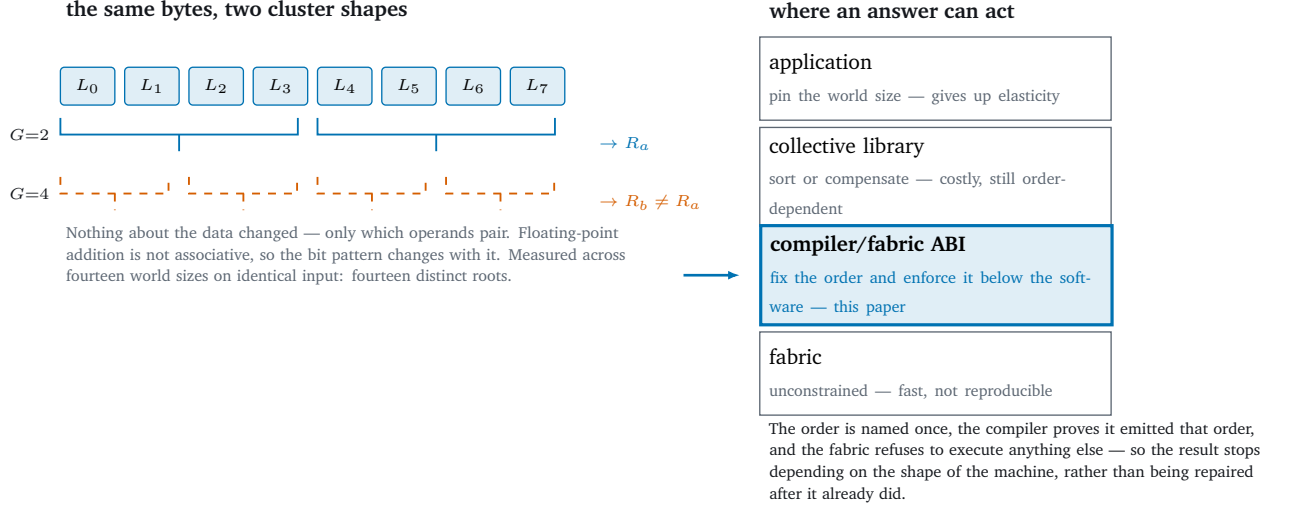


Figure 1. What cluster reshape does to a reduction, and where an answer can act. Only the grouping changes; because floating-point addition is not associative, the bit pattern changes with it. Measured across fourteen world sizes on identical input, fourteen distinct roots. Existing answers act above the fabric and pay for it in elasticity or in cost; this paper fixes the order at the compiler/fabric interface, below the software that could violate it.

answer: for two orders σ, σ' of the same multiset [?],

$$\left| \sum_{\sigma} x_i - \sum_{\sigma'} x_i \right| \leq (N-1)u \sum_i |x_i| + \mathcal{O}(u^2), \quad (1)$$

a bound that is catastrophically loose under cancellation, and whose consequences are documented across HPC and deep-learning workloads [?].

The consequence is measured in §7.1, not asserted: reducing identical input *bytes* at fourteen rank counts under PyTorch’s distributed collectives [?] — same data, same seed, same code, only the shape of the job changing — returns fourteen distinct 64-bit results, while repeated runs at any *fixed* shape are bitwise identical five times out of five. Reshape moves the answer; noise does not. In a training loop the divergence compounds: the reduced gradient becomes the next step’s weights. Bit-for-bit regression testing fails the same way — a code accepted on a development cluster produces different last bits on the production cluster, and the reviewer cannot separate a new bug from the reshape artefact. In regulated deployments, inability to replay a computation bitwise across hardware generations is increasingly disqualifying.

Two sources produce this. *Order-dependence of the reducer*: NCCL selects its (algorithm, protocol) pair at communicator initialisation from a cost model over architecture, node and rank counts and message size [?]; the data are never consulted, so with all randomness suppressed the algorithm remains a deterministic function of $(G, \text{topology})$. *Reassociation inside the reducer*: pipeline overlap, SIMD lanes and message merging are each an implicit reassociation.

Both have been attacked before, along one axis at a time (§10).

What no line of work states is the invariant *at the interface*, enforced below the software that could violate it. A library flag cannot forbid Source 1, because the library has no authority over what cluster size is launched tomorrow; a fabric guarantee alone cannot forbid Source 2, because the fabric cannot see the tree a scheduler intends. The discipline has to be a contract shared by both. The principle is not new — IEEE 754-2019 already makes reproducibility an obligation on language processors, forbidding reordering and fusion under its *reproducible* attribute [?] — but that obligation binds within one program and says nothing about how many workers run a collective or how they are wired. We extend it to the interface the standard does not reach, and index it by problem size so that it survives a change of deployment.

Contributions. **(C1)** Reduction order as a size-indexed compiler/fabric ABI, with a structural invariance argument (§3): Claim 1 (cross-topology bitwise invariance) and Property 1 (presence-gated fail-stop), connected by Proposition 1. We claim the framing, not the difficulty of the proof: once operand binding is a function of tree-node identity alone, invariance follows by unwinding definitions, and we say so rather than dressing it as a discovery. The work is in the three places the proof does not reach — building a fabric that enforces the binding with no silently-wrong failure mode, establishing that every remaining degree of freedom in a real compiler is orthogonal to it, and carrying the discipline onto a production collective and paying its price. **(C2)** A fabric (§4) and compiler (§5) that jointly realise it, in which topology is a runtime table rather than an RTL artefact and one compiler pass *constructs* the single-writer discipline the argument needs. **(C3)** A hardware campaign at scale and a structural pricing of the ABI (§§6, 7), plus a software export onto production collectives (§8). What is *not* claimed: interoperability with any external reducer, strict IEEE 754 (§3.6), silicon timing, or novel component algorithms — our eighteen assignment and ten mapping algorithms are prior art [?, ?, ?]; the observation is that any choice among them inherits invariance by composition.

Roadmap. §2 fixes the threat model and places the work against prior reproducibility approaches. §3 is the contract: canonical tree, operand binding, the fifteen assumptions, Claim 1, Property 1 and the orthogonality proposition. §4 is the hardware that establishes the fabric-side invariants and §5 the compiler that discharges the compiler-side obligations. §§6–7 are the campaign in which the two are run together; §8 carries the same discipline onto production collectives in software and prices it there; §9 is the analysis, scope and generalisation.

2 Background and Threat Model

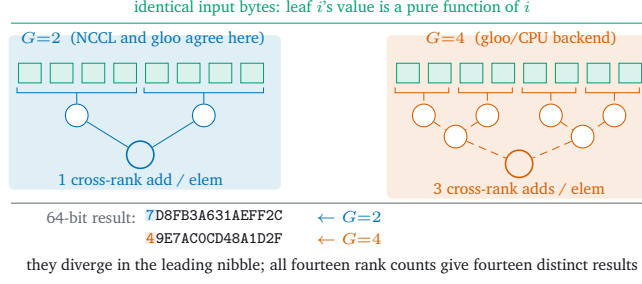


Figure 2. The premise, measured rather than assumed. Reducing identical bytes at different world sizes yields different 64-bit roots. Above $G=2$ the sweep runs on the gloo/CPU backend, because NCCL requires one rank per device and the node has two GPUs.

Where the order comes from. Let τ be a binary reduction tree over N binary64 values and Σ_τ the 64-bit result of evaluating it with RNE rounding at every internal node. Equation (1) is loose generically and tight under cancellation: when the exact sum is small relative to the partial sums, Σ_τ and $\Sigma_{\tau'}$ can differ at the exponent scale of the largest partial sum rather than of the result. Production collectives choose τ from the deployment. Ring all-reduce [?] orders along a physical worker ring; recursive halving–doubling [?] orders by a power-of-two recursive pattern; double-binary-tree all-reduce [?] interleaves two trees; runtime-hybrid libraries [?, ?] select among these per message size from topology discovered at communicator initialisation. In each case τ is a function of $(G, \text{topology}, \text{message size})$ and never of the data, so reshaping the cluster reshapes τ and therefore Σ_τ . The same dependence is documented for MPI reduction operations [?].

Definition 1 (Cross-topology bitwise reproducibility). Let $L \in (\text{bits}_{64})^N$ be a fixed leaf-value vector. For every $(G, \text{topology})$ with topology in $\mathcal{T}$, every valid SPRS compilation and every valid execution of the fabric, the observed 64-bit root is the same sequence of bits.

$\mathcal{T}$ is the family of flat direct networks realisable by a configuration of the per-router adjacency and routing tables: linear, ring, 2D mesh, 2D torus, hypercube and fat-tree. Definition 1 is *structural* — a statement about what the compiler/fabric pair can be shown to guarantee, not an empirical average over executions — which is why the evaluation reports a divergence count rather than a divergence rate.

Four things are deliberately outside it. (i) Agreement between SPRS and an external reducer that fixes a different canonical order. (ii) Tolerance to single-event upsets or transient faults. (iii) Portability of bit patterns between fabrics with different FADD microarchitectures. (iv) Adversarial or merely unlucky traffic patterns affecting *cycle counts*; these cannot affect the bit result, and

Table 1. Reproducibility approaches, by what fixes the order, what the result is then invariant under, and whether the FP64 operator survives. Source 1 = algorithm selection as a function of $(G, \text{topology})$; Source 2 = reassociation inside the algorithm. $\checkmark$ = the source cannot change the bits within the stated scope; *vac.* = made vacuous by an order-independent operator rather than constrained. No column is ours alone; the last row is the conjunction.

Approach	Order-fixing mechanism			Invariant across	Src 1	Src 2	FP64 op.
ReproBLAS [?]	Binned	accumulator	(order-independent)	Processor count, partition, MPI tree shape	$\checkmark$ vac.	$\checkmark$	no
ExBLAS [?, ?]	Long accumulator + FP expansions			Processor count, partition; two clusters	$\checkmark$ vac.	$\checkmark$	no
Kahan / pairwise [?, ?]	Compensated arithmetic			Nothing: error is reduced, bits are not	—	$\sim^\dagger$	yes
oneMKL strict CNR [?]	Fixed operation order in a shipping BLAS			Thread count, input partitioning; one node	$\checkmark$	$\checkmark$	yes
TBIK [?]	One fixed tree, intra- + inter-GPU			Tensor-parallel size; power-of-two, one node	$\checkmark$	$\checkmark$	yes
Pinned collectives [?, ?]	Fixed algorithm + single channel			One $(G, \text{topology})^*$	$\checkmark^*$	$\checkmark$	yes
SHARP [?, ?]	Operand order pinned in the switch			One application-to-system mapping [*]	$\checkmark^*$	$\checkmark$	yes
Groq TSP [?]	Software-scheduled routing, no arbitration			One compiled system configuration [‡]	$\sim^\ddagger$	—	n/a
SPRS (this paper)	Size-indexed	FBT(N)	ABI,	All $(G, \text{topology}) \in \mathcal{T}$; assignment, placement, routing, schedule	$\checkmark$	$\checkmark$	yes
	scoreboard-gated in RTL						

[†] Reduces the error bound, not the bit pattern: the same values in a different order, or with a different number of lane accumulators, still differ in the last bit. ^{*} Within one deployment only — SHARP is reproducible “for a given application-to-system mapping” [?], and NCCL selects its algorithm from rank and node counts. [‡] Schedule determinism only: no bitwise claim is made, and the all-reduce is keyed to the packaging hierarchy.

§4.6 says what they can affect.

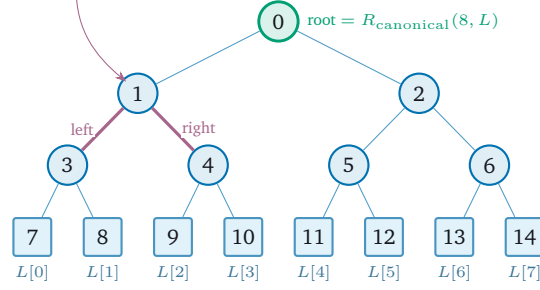
Table 1 classifies the field along those two sources. Two groups already deliver invariance under a change of resource count and we do not claim otherwise: order-independent accumulators do it by changing the operator, and fixed-order schemes do it with the operator intact but each within one library and along one axis. What no row other than the last combines is invariance across network *topologies* rather than resource counts alone, an unmodified FP64 operator, arbitrary non-power-of-two N , and enforcement below the software stack with a fail-stop failure mode. §10 treats the rows individually.

3 The Reduction ABI

3.1 The Canonical Tree

Definition 2 (Canonical full binary tree). For $N \geq 1$, $\text{FBT}(N)$ is the heap-indexed binary tree with $2N - 1$ nodes: root at index 0, children of i at $2i + 1$ and $2i + 2$; internal nodes $\{0, \dots, N - 2\}$, leaves $\{N - 1, \dots, 2N - 2\}$. Since

operand binding at $v=1$ (Def. 3): exactly one COMPUTE, with $\text{addr_a}=\text{dmem}(3)$ and $\text{addr_b}=\text{dmem}(4)$ — a pure function of the node v . Neither G , nor topology, mapping, refinement or schedule appears in it.



FBT(N) has $2N-1$ nodes; children of n at $2n+1$ and $2n+2$; internal $\{0, \dots, N-2\}$ are circles and leaves $\{N-1, \dots, 2N-2\}$ squares, so every internal node has exactly two children.

Leaf enumeration $\text{idx}(n) = n - (N-1)$: the k -th smallest heap index takes $L[k]$. That is left-to-right visual order only when N is a power of two.

Internal post-order $\pi_8 = \langle 3, 4, 1, 5, 6, 2, 0 \rangle$; campaign root at $N=8$ is 0x40E1948E978D4FDF (Table I).

Figure 3. FBT(N): heap-indexed, leaves at $N-1 \dots 2N-2$ in ascending index order. The tree, not the machine, defines the result.

$2N-1$ is odd, every internal node has exactly two children: the single-child edge case is eliminated at the tree level.

Definition 3 (Canonical leaf enumeration). idx assigns index k to the k -th smallest heap index in the leaf set of FBT(N).

Definition 3 needs stating precisely because the obvious paraphrase is wrong. Ascending heap index and left-to-right visual order coincide exactly when N is a power of two. Otherwise FBT(N) is depth-imbalanced and a shallower leaf carries a *smaller* heap index than deeper leaves to its left: for $N=7$ the heap order is $\langle 6, \dots, 12 \rangle$ while visual order is $\langle 7, \dots, 12, 6 \rangle$. Seven of the sixteen leaf counts we evaluate are of this shape, and at $N=200$ the two enumerations give roots differing by one ulp. An implementation using visual order implements a *different* contract.

Write π_N for the post-order traversal of FBT(N) restricted to internal nodes — a pure function of N ; $\pi_4 = \langle 1, 2, 0 \rangle$. The canonical evaluation is

$$R_{\text{canonical}}(N, L) = \text{evaluate}(\text{FBT}(N), L, \text{fadd}_{\text{rtl}}), \quad (2)$$

with L indexed by idx and fadd_{rtl} the combinational semantics of our FP64 adder (§3.6). Equation (2) is the *target* of every SPRS execution on (N, L) .

The canonical order is a constraint, not a tautology. One might suspect that any association order yields one root per N , so agreement across topologies measures nothing. We measured how far that is from true. On the leaf vectors evaluated here, right-to-left sequential summation differs from $R_{\text{canonical}}$ by up to 55 ulp at $N=8192$, and left-to-right differs at twelve of the sixteen leaf counts. The discriminating power is strongly N -dependent and not monotone: exhaustively at $N=8$, all 40,320 leaf permutations yield only three distinct

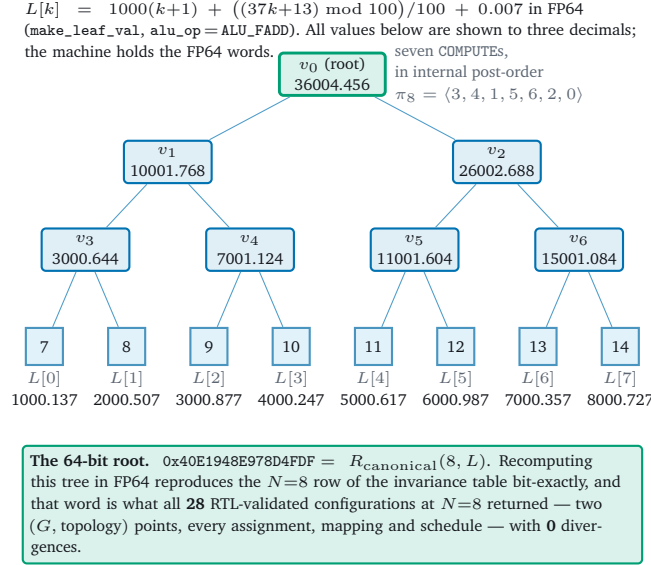


Figure 4. A worked reduction at $N=8$ with the real leaf values, showing the canonical root the campaign reports for that leaf count. The shape of the machine appears nowhere in the computation.

roots, 88.6% of them the canonical one, so a uniformly random leaf-order violation is detected with probability 0.114; at $N = 16$, all 9,694,845 in-order tree shapes yield four distinct roots spanning two ulp. Across the sixteen leaf counts the probability that a sampled reassociation moves the root ranges from 0.08 to 0.90. Two consequences follow and we adopt both: the oracle is non-vacuous, so an agreeing run is evidence; and the rows of the invariance table are not equally informative — at small N agreement is close to free, and the weight of the evidence sits at the large leaf counts.

3.2 The ABI

Definition 4 (Operand binding). For every internal node v of $\text{FBT}(N)$ the compiler emits exactly one **COMPUTE** on the PE hosting v , with $\text{addr_a} = \text{dmem}(\text{left}(v))$ and $\text{addr_b} = \text{dmem}(\text{right}(v))$, where $\text{dmem}(\cdot)$ is a per-PE address assigned by the allocator. The binding is a pure function of v .

Definition 5 (Leaf binding). For every leaf k , the **GENERATE** on the PE hosting k writes $L[\text{idx}(k)]$ into the DMEM slot bound to k .

The fabric executes the two disciplines symmetrically: **COMPUTE** reads both DMEM slots at one clock edge, scoreboard-gated, and writes $\text{fadd}_{\text{rtl}}(\cdot, \cdot)$; **GENERATE** delivers $L[\text{idx}(k)]$ with the scoreboard set. Neither discipline references G , topology, assignment α , mapping μ , refinement r , or timing. These are left free precisely so that schedulers may keep optimising without touching the contract, and they are what the campaign of §6 searches over.

3.3 Assumptions

Fifteen assumptions carry the two properties: five *compiler obligations* (C-*), discharged by the compiler or the user, and ten *fabric invariants* (F-*), established by RTL inspection. Eleven are hypotheses of Claim 1; the four marked † are liveness invariants that can stop a run but cannot alter a root the fabric does produce, and are the hypotheses of Property 1 instead. The supplemental material states all fifteen. The partition is not cosmetic: the C-* obligations are post-conditions on the emitted image, checkable without executing anything, while the F-* invariants are properties of the fabric independent of any compilation — so a failure of Claim 1 is diagnosable as a compiler bug or an RTL bug, not both.

Three conditions a reader might expect are absent. *Compilation determinism* is not a hypothesis: Claim 1 quantifies over every image the compiler can emit, and a statement true of all of them does not need the compiler to be a function of its seed. *Partition connectedness* is not either, although 2,061 of the RTL-validated configurations (73.0%) have partitions that are not connected subtrees: operand binding is by node identity and every cross-PE child gets a dedicated receive slot, so the emitted binding is $(\text{left}(v), \text{right}(v))$ for any assignment — checked statically on 1,010,895 COMPUTE and 731,982 SEND instructions across 681 configurations, with zero violations. What disconnection costs is DMEM occupancy, and that is what C-mem bounds. The *identity slot* DMEM[0] is reserved but unread here, since FBT(N) has no single-child internal node.

Remark 1 (The C-mem margin, and what lies past it). C-mem is the one hypothesis of Claim 1 the campaign came close to violating, so we report the margin rather than the fact of compliance. Peak DMEM occupancy over the validated configurations reaches 491 of 512 slots, a margin of 21. Past the ceiling the claim fails, and it fails *silently*: on a constructed FBT(512) instance at $G = 2$ with all leaves on one PE, peak occupancy is 522 slots, 11 emitted addresses exceed the depth, and the run terminates normally — no P_ERROR, no deadlock — returning a wrong root, and two *different* wrong roots under two different packet-delivery orders. Aliasing defeats the presence gate rather than tripping it, because the aliased slot has been written by something. The fabric cannot detect the condition, which is why C-mem is a hypothesis of Claim 1 rather than a property of the machine. The compiler can: the emitter now carries a post-condition over the instruction words it is about to write and raises EmitRefused rather than emit an image whose addresses would alias, so the failure above is a recorded compile failure in the released tool and a wrong root only in the version we measured it on. No configuration in the campaign trips that post-condition, which is the same statement as the margin.

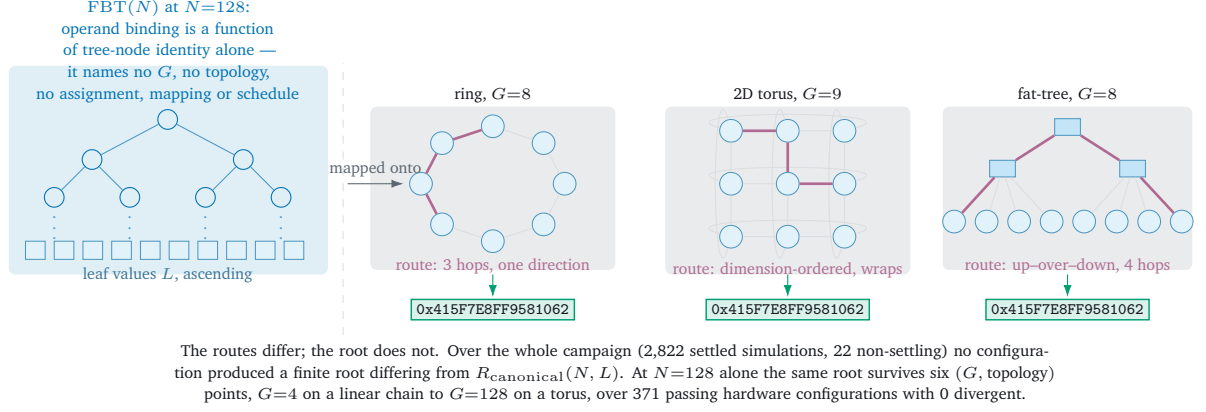


Figure 5. Claim 1. The same reduction DAG mapped onto different topologies yields the identical 64-bit root: routes differ, the root does not.

3.4 Claim 1 and the Fail-Stop Property

Claim 1 (Cross-topology bitwise invariance). Let $N \in \mathbb{N}$, let $L \in (\text{bits}_{64})^N$ be a fixed leaf-value vector, and let $\mathcal{T}$ be the family of flat direct-network topologies realisable by adjacency tables consistent with `noc_system.sv`. Assume C-sw, C-bind, C-leaf, C-mem and F-sb-mono, F-sb-gate, F-fw, F-inorder, F-gci-haz, F-arb, F-fadd. Let G be a worker count and $\tau \in \mathcal{T}$ a topology for which the compiler emits an image satisfying C-mem; let α, μ, r be any valid assignment, mapping and refinement, and s the emitted per-PE schedule. If the run terminates normally — `all_done` asserted, no `P_ERROR`, no watchdog — then the observed 64-bit root, the result of the root PE’s final `COMPUTE`, satisfies

$$R_{\text{observed}}(N, L, G, \tau, \alpha, \mu, r, s) = R_{\text{canonical}}(N, L). \quad (3)$$

Three features of the statement bind a reimplementer. *First*, C-mem is a compilability precondition, not a decoration: the claim asserts nothing about a configuration whose image exceeds the per-PE DMEM budget or whose router count exceeds what the adjacency interface can address. One instance in our suite is in that position, and it is also the one that never settles (§11). *Second*, the claim is partial-correctness; liveness is Property 1’s business, which is exactly why C-route, F-credit, F-misroute and F-watchdog are not hypotheses here. *Third*, the observable is the root PE’s final `COMPUTE` result, not the root DMEM slot, which no emitted instruction ever reads back.

Property 1 (Presence-gated fail-stop). Suppose an assumption violation causes some `COMPUTE` to name a DMEM address whose presence bit is never set — a misrouted and dropped packet (F-misroute), a lost credit (F-credit), an unreachable destination (C-route), or an operand bound to a slot no agent writes (C-bind). Then no instruction retires on the affected PE, F-watchdog fires after `WATCHDOG_MAX` = 20,000 consecutive non-retiring cycles, and the PE enters

P_ERROR recording error_pc and error_state; all_done is never asserted and no root is produced. An invalid opcode raises P_ERROR immediately.

The converse bound is equally explicit, because the property is easy to over-read. The fabric checks presence, not identity: a COMPUTE whose operands are both present executes, whatever they contain. Violations leaving every read address present — an operand misbound to a live slot, a corrupted leaf value, a duplicate write to a receive buffer, an over-budget image in which two addresses alias onto one written slot, or a violation of any of the seven safety F-* invariants — terminate normally with a wrong root. The fabric does not detect these and we do not claim it does; they are discharged by the compile-time post-condition on the emitted image and by comparison of the final root against the oracle.

How the hypotheses carry the claim. Four steps, each of the eleven used at least once. *Addresses* (C-bind, C-mem): build_schedule binds a COMPUTE's operands from an address map keyed on node identity, and every cross-PE child holds a dedicated receive slot, so the emitted binding is $(\text{left}(v), \text{right}(v))$ under any assignment; C-mem is what promotes that address-level statement to a value-level one, since the fabric truncates addresses and distinct addresses stay distinct only while the image fits DMEM. *Uniqueness* (C-sw, F-sb-mono): $\text{FBT}(N)$ is a tree, so each node has one consumer; the allocator therefore gives each receive buffer exactly one writer, and its scoreboard bit transitions $0 \rightarrow 1$ exactly once per program. *Values* (F-sb-gate, F-fw, F-inorder, F-gci-haz, F-arb): every address a COMPUTE reads is a receive buffer or a local temporary. For the first, F-sb-gate holds the instruction in DE until both bits are set, so at the EX edge both slots hold their committed remote writes, with F-fw closing the BRAM read-first race. For the second the scoreboard is silent — a recycled slot's bit is already set — and need not speak: under F-inorder the overwriting COMPUTE samples its operands strictly before writeback, and the one writer program order does not sequence, an in-flight GENERATE, is covered by F-gci-haz and its retry latch F-arb. *Function* (F-fadd): the committed value is fadd_{rtl} of those stable inputs, a pure function. Composing, every internal DMEM value at termination is fadd_{rtl} applied to v 's two children — with leaves supplied by C-leaf — so the root PE's final COMPUTE returns $\text{evaluate}(\text{FBT}(N), L, \text{fadd}_{\text{rtl}}) = R_{\text{canonical}}(N, L)$, in which no free axis appears.

We label these a *claim* and a *property* rather than theorems because the supporting argument proceeds by RTL inspection, a compiler post-condition, end-to-end root comparison and mutation testing, not by formal proof (§11, L1).

3.5 Mapping/Topology Orthogonality

Of the axes Claim 1 quantifies over, (G, α) is where the substantive work lives: changing G changes the number of PEs and changing α changes which PE hosts which node. The $(\text{topology}, \mu, \rho, s)$ slice admits a sharper statement.

Proposition 1 (Mapping/topology orthogonality). Fix N, L, α, r and the FADD unit. For any two valid mappings μ_1, μ_2 and any two valid routing tables ρ_1, ρ_2 for (possibly different) topologies in $\mathcal{T}$,

$$R_{\text{observed}}(\mu_1, \rho_1) = R_{\text{observed}}(\mu_2, \rho_2) = R_{\text{canonical}}(N, L). \quad (4)$$

Structural argument. By inspection of `build_schedule` and `generate_instructions`: a COMPUTE’s $(\text{addr_a}, \text{addr_b}, \text{dest})$ triple depends only on α and the DMEM allocator’s output, neither of which reads μ or the routing tables. A SEND’s $(\text{dest_gpu}, \text{target_addr}, \text{link})$ triple does depend on μ and on routing, but those determine only *where* data flows, not the operand binding at the receiving PE. Under F-sb-gate the receiving COMPUTE reads DMEM by address, not by arrival time, port or VC. Hence R_{observed} is independent of (μ, ρ) . $\square$

Proposition 1 is the cleanest statement of what the ABI buys: the two things that change most obviously under cluster reshape are structurally orthogonal to the bit result. This is a consequence of the code path that emits the instructions, not a hypothesis to be validated empirically.

3.6 FADD Policy

Our adder (`fp64_add.sv`, with a bit-exact Python mirror) implements a *deterministic subset* of IEEE 754-2019 binary64 addition, which we specify fully as *FTZ+RNE*: subnormal inputs flushed to ± 0 preserving sign; round-to-nearest-even only, with no rounding-mode register; post-round subnormals flushed likewise; $x + (-x) = +0$ and $(-0) + (-0) = +0$ (a documented deviation from §6.3 of the standard); a quieted qNaN whose sign and payload are taken from a when a is NaN and from b otherwise; and the usual infinity rules with $\pm\infty + \mp\infty$ returning the canonical qNaN. Invariance depends only on `fadd_rtl` being a pure function of its inputs, not on any particular IEEE 754 feature; a subnormal-aware alternative inherits Claim 1 by construction. The subset is checked rather than asserted: the shipped adder shows zero disagreements with correctly-rounded IEEE 754 addition over 4,644,000 vectors inside the documented scope. The adder we first submitted did not meet the RNE clause on effective subtraction. §4.8 reports the defect and its repair, and §11 (L13) reports why neither the campaign nor the software mirror could have found it — a finding about the evaluation method rather than an erratum.

4 The Fabric

The fabric establishes the F^* invariants. It supports up to 4096 PEs in a single Verilog instantiation, and four properties carry the ABI into hardware: topology is a runtime configuration rather than an RTL artefact (§4.1); the ISA is small enough to audit for ABI adherence (§4.2); the scoreboard is the single synchronisation point across the three agents that write DMEM, and program order covers the one case it cannot (§§4.3–4.4); and every violation leaving an operand absent halts at a diagnostic state rather than corrupting silently (§4.5).

4.1 Topology as a Runtime Parameter

Two compiler-loaded per-router tables parameterise the network. The *adjacency table* maps (router, port) $\rightarrow$ (target router, target port) | ADJ_DISCONNECTED; the *routing table* maps (router, destination) $\rightarrow$ output port. Both are written through a configuration interface before `program_start`. A single `noc_system` module supports the whole family $\mathcal{T}$; port 0 is reserved for the local network interface on every router, the one structural convention independent of topology. Adding a topology therefore requires a Python classmethod that emits the two tables and no Verilog change: the compiler ships six families (linear, ring, 2D mesh, 2D torus, fat-tree, hypercube) with $O(1)$ analytical shortest paths and a BFS fallback at $O(G)$ per source for custom networks.

The binding ceiling on cluster size is the width of that configuration interface rather than a design parameter. Adjacency writes carry a 12-bit router id whose top code is the disconnected sentinel, so ids are addressable to 4,094 and a configuration above that aliases silently — which is exactly what defeats one instance of the suite (§11, L0). `MAX_ROUTERS` and `MAX_PORTS` are declared in the packages and referenced by no module; they enforce nothing, and we say so because a reader auditing the RTL for a cluster-size bound will find them first.

4.2 Packet Format and Reduction ISA

Packets are 96-bit and single-flit, with the field layout given in Fig. 7. Bit [64] is read by the router but written as the literal `6'b0` by the only packet-forming assignment in the design, so buffer occupancy cannot be perturbed by the payload and the cycle counts we report are value-agnostic by construction; §4.6 says what that costs. The instruction word is 64 bits,

$$\underbrace{\text{opcode}}_4 \parallel \underbrace{\text{aux}}_4 \parallel \underbrace{\text{dest}}_{16} \parallel \underbrace{\text{addr_a}}_{16} \parallel \underbrace{\text{addr_b}}_{16} \parallel \underbrace{\text{reserved}}_8 \text{ bits,}$$

the 64-bit instruction word

■ the two fields operand binding is written in — C-bind is exactly their value

[63:60] opcode	[59:56] aux	[55:40] dest write-back address	[39:24] addr_a left operand	[23:8] addr_b right operand	[7:0] reserved
-------------------	----------------	------------------------------------	--------------------------------	--------------------------------	-------------------

opcodes

GENERATE writes a leaf value to `dest`. COMPUTE reads `addr_a` and `addr_b`, adds them in the FP64 unit, and writes `dest` — exactly two operands, which is why no same-fabric reducer can be shallower than $\text{FBT}(N)$. SEND carries the value at `addr_a` to the worker named in `dest`.

the 96-bit packet a SEND becomes

[95:80] dest_gpu_id	[79:70] target_addr	[69:64] resv.	[63:0] payload — the 64-bit value
------------------------	------------------------	------------------	-----------------------------------

■ The instruction's 16-bit `dest` is carried on the wire in only ten bits, while DMEM is 512 deep. An image naming more addresses than that on one worker aliases two onto one slot with no flag raised — which is why C-mem is a compile-time obligation and not a run-time check.

Figure 6. The two encodings the ABI rides on. Operand binding is not a convention but two address fields in the instruction word; C-mem is the gap between the 16-bit `dest` and the ten bits that carry it.

Table 2. The four live opcodes. Opcodes $\geq 4'h4$, other than the one reserved slot, trap to `P_ERROR` immediately.

Mnemonic	Code	Effect
NOP	4'h0	Consume one slot.
GENERATE	4'h1	Fetch a leaf into <code>DMEM[dest]</code> via the GCI interface; set <code>scoreboard[dest]</code> .
COMPUTE	4'h2	Read <code>DMEM[addr_a]</code> , <code>DMEM[addr_b]</code> (scoreboard-gated), write <code>alu_op(a, b)</code> to <code>DMEM[dest]</code> , set <code>scoreboard[dest]</code> .
SEND	4'h3	Enqueue an RDMA-push packet with payload <code>DMEM[src]</code> to <code>(dest_gpu, target_addr)</code> on link.
OP_RESERVED	4'h4	Reserved for a future BARRIER/RESET_SB.

with four live opcodes (Table 2). Two source fields are mandatory because COMPUTE names both members of the operand pair the ABI binds; that is the ISA-level expression of Def. 4. For SEND, `dest` is the *destination-side* DMEM address carried as `target_addr` in the packet, `addr_a` the local source and `addr_b` the destination PE. Egress port is deliberately *not* an instruction field: no small fixed field could address a router's egress ports, which range to $P = 66$ across our suite (§4.7), so the port is resolved in hardware by the emitting router's route-table lookup. That single decision is what keeps topology a configuration input rather than a property of the emitted code: the instruction stream names logical destinations, and only the adjacency and routing tables name the network.

The identity slot `DMEM[0]` is realised by resetting DMEM to all-zero with `scoreboard[0]` set — zero being the identity for ADD, FADD, OR and XOR —

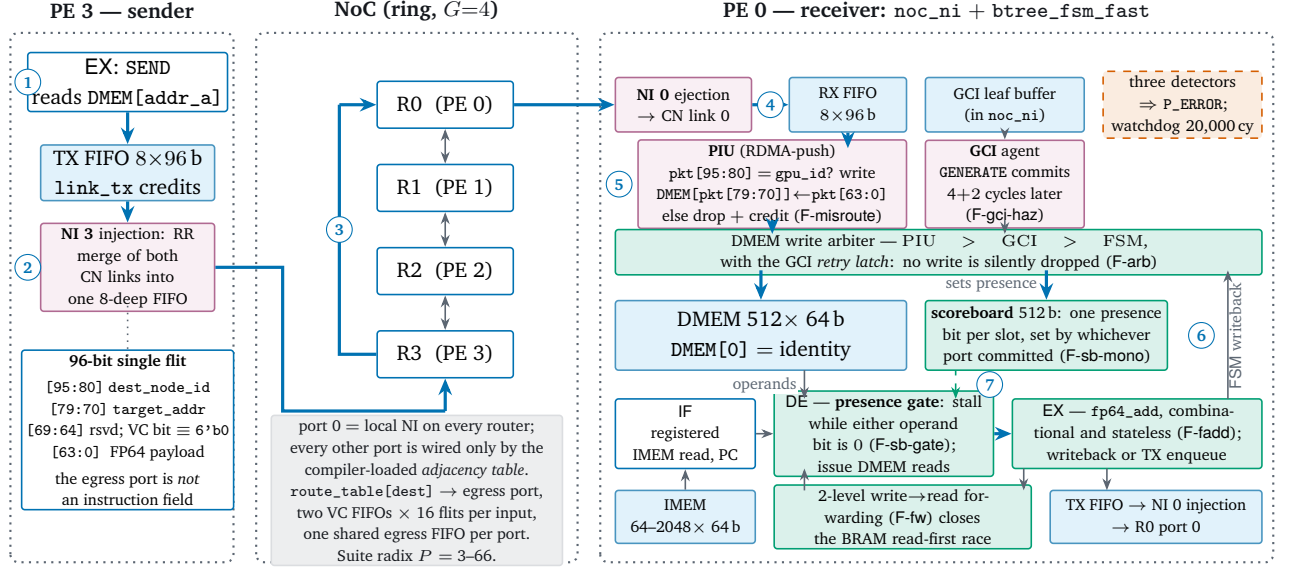


Figure 7. The fabric, traced along the path one 64-bit operand actually takes: a SEND on PE 3 forms a 96-bit single flit, the network interface merges the two compute-node links into one injection FIFO, the router resolves the egress port from its compiler-loaded route table, and on arrival the PIU — not a RECV instruction, of which there is none — checks `dest_gpu_id` and writes DMEM itself, dropping and returning credit on a mismatch (F-misroute). The one thing to take from it is the last two steps: the scoreboard is the single source of truth for “data present at address a ” across all three agents that write DMEM (PIU, GCI, FSM writeback), and DE holds the dependent COMPUTE until both operand bits are set, so a scoreboard-gated read is race-free against arrival time. The write arbiter’s GCI retry latch is why no write is silently dropped, and §4.4 shows the latch is necessary for the *value*, not only for liveness.

so that asymmetric trees or variable-arity reductions could supply an identity operand without an ISA change. It is unread here.

4.3 Pipeline, Scoreboard and Forwarding

Each PE runs the three-stage pipeline of Fig. 7, with EX additionally stalling on back-pressure or a write collision. The DMEM read path carries a two-level write-to-read forwarding network realising F-fw: level 1 forwards a cycle- t write to a cycle- $(t+1)$ read of the same address, the back-to-back COMPUTE case; level 2 forwards across a single intervening stall, where the one-cycle forward has expired before the read settles. Given the pipeline depth and the scoreboard’s one-cycle gate the two levels exhaust the races.

Because presence is decided in one place independently of which port delivered

the data, a scoreboard-gated read is structurally race-free against remote arrival — the fabric never needs to know *when* a packet lands, only that it has.

4.4 Why the Scoreboard Alone Is Not Enough

The single-writer discipline C-sw is scoped to receive buffers, and that scoping is the part of the design most easily misread, so we state what covers the rest. The allocator returns a child's slot to a free list once its parent has consumed it (§5), so a *local temporary* can be written more than once per program. Read against a scoreboard bit that is never cleared, that looks unsafe: the second writer finds the bit already set. Nor is reuse a corner case one could restrict the claim away from — over 562 compiled configurations we count 108,607 recycled addresses and 397,422 reuse events, and *every* configuration recycles.

It is safe, and there are exactly two cases, separated by how many readers of the old value are still outstanding when the slot is rewritten. In 31.3% of reuse events there are none: the last consumer has retired and the slot was only then returned to the free list. In the remaining 68.7% there is exactly one, and it is the very COMPUTE that overwrites the slot, which under F-inorder samples both operands at the DE read port strictly before writing back in EX. Over the same configurations no reuse event has a remaining reader other than the overwriting instruction and none has two, so the two cases are exhaustive (zero unsafe events in 397,422). The ordering is supplied by program order on one PE, not by the scoreboard. The scoreboard exists for the one case program order cannot cover — a value written by a *different* PE, whose arrival time is a function of routing and contention — and those destinations are precisely the receive buffers, which are never recycled. We verified the separation directly on the emitted images: the recycled and the remotely-written address sets never intersect.

One sub-case of the second is not covered by program order either, and it is why F-gci-haz and F-arb are hypotheses of Claim 1 rather than liveness conveniences. GENERATE is asynchronous: the instruction asserts `gci_start` in EX and the GCI agent commits the leaf 4 or more cycles later, against a compiler-emitted gap of 6 cycles at minimum and 12 at the median. 35.4% of GENERATEs (29,097 of 82,084) target a *recycled* slot whose bit is already set by the previous writer, and every one of them is followed by a read of that slot; the scoreboard therefore cannot stall the consumer and cannot be what makes them safe. The in-flight interlock F-gci-haz, which stalls in DE any COMPUTE or SEND naming a pending destination, is what makes them safe. The RTL comment calls that interlock belt-and-braces with the scoreboard; it is not.

F-arb completes the pair. The DMEM write arbiter is strict-priority PIU > GCI > FSM — PIU always wins, preserving the RDMA-push semantic that in-

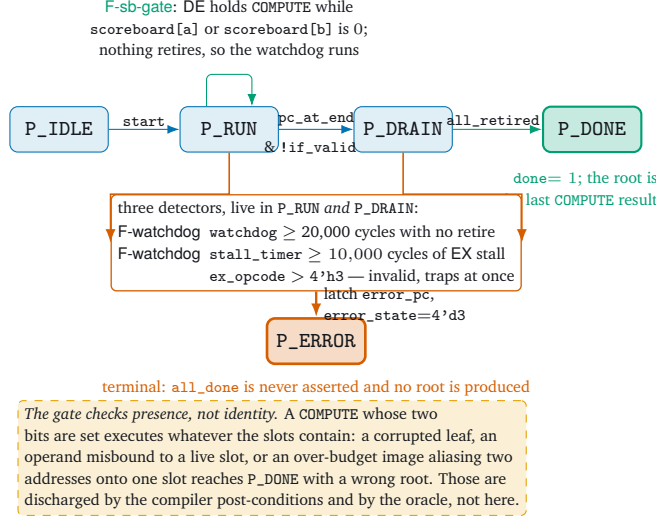


Figure 8. Property 2. A missing or misrouted operand halts the run; it never silently produces a different root.

coming data is never blocked — with a GCI *retry latch*: when a GCI completion collides with a PIU write in the same cycle the arbiter drops the GCI write, and the latch captures the pending value and re-asserts it until accepted. Without the latch the leaf value is silently lost while the PIU still sets the presence bit, and the next COMPUTE reads a wrong leaf and produces a wrong root with no diagnostic. Remove either mechanism and the consumer reads a stale value, with no stall and no error: they are necessary for the value, not merely for liveness, which is the reason both appear in Claim 1’s hypothesis list where a reader might expect only F-sb-gate.

4.5 Fail-Stop Discipline

The PIU’s header check against the local PE (Fig. 7) is what discards a misrouted packet — a routing-table bug, a corrupted packet, a torn adjacency — rather than writing it (F-misroute). The drop and the credit return are real hardware; the misroute flag itself is an internal signal with no reader and no module port, so a misroute is observable only through the liveness failure it causes. That is the intended shape: a bad routing table leaves the destination’s presence bit clear, its dependent COMPUTE stalls, and the run halts with a recorded program counter instead of returning a different number.

Three detectors run in the P_RUN state. The watchdog fires after WATCHDOG_MAX = 20,000 cycles with no instruction retired; a stall timer fires after TIMEOUT_MAX = 10,000 cycles of continuous EX stall; and any opcode above the reserved slot traps immediately. Each raises P_ERROR with error_pc and error_state for post-mortem. The termination outcomes are therefore (i) a correct result, (ii) P_ERROR with diagnostic context, or (iii) a watchdog timeout with the same context, and no configuration in the campaign produced a fourth. We do not

claim a fourth is impossible: Remark 1 gives the one construction — an image over the C-mem budget — that defeats the presence gate instead of tripping it.

4.6 Flow Control, Routing, and Deadlock

Each input port carries two virtual channels and every emitted testbench sizes router FIFOs at 16 flits, overriding the RTL default of 8. Per-link credit-based flow control prevents ingress overflow, with credit returns serialised through a pending-credit register that drains one pulse per cycle; that register is two bits wide and wraps only on a backlog of four, which requires two virtual channels to be read in one cycle. It never happens here, for the reason the next paragraph gives, and its value is provably zero in every reported cycle.

One channel, and no escape. The router reads bit [64] as a VC selector but nothing writes it: the field is a constant in the only packet-forming assignment, the ISA has no VC field, and the compiler contains no virtual-channel logic, so every packet in every reported run travels on VC0. Nor could the second channel serve as an escape channel if the compiler did assign one — the router's egress buffer is a single FIFO per port shared across VCs, so the two channels are coupled at the output whatever the header says.

What discharges C-route, and what does not. The compiler computes routing tables by greedy minimal-distance search over the adjacency list; they are not dimension-ordered. That construction gives exactly what C-route claims: every emitted path terminates at the destination named in the header and visits no router twice. It does not give an acyclic channel-dependency graph in the sense of Dally and Seitz [?]. Built directly from the emitted tables, the CDG contains cycles on every ring instance (cycle length G , the textbook bidirectional-ring cycle) and on every torus instance (length 6–32); it is acyclic on linear, mesh, hypercube and fat-tree at every suite size. We therefore claim no deadlock-freedom property on cyclic topologies. Deadlock is possible in principle, and the fabric converts it into a fail-stop rather than a wrong answer: a deadlocked PE retires nothing, the watchdog fires, and Property 1 applies. That is where most of the campaign's non-settling runs come from (§11, L0a). A compiler-side dimension-ordered pass with a VC-escape assignment, and a per-VC egress buffer to make that escape real, are the two changes a production deployment would need; neither can change a bit result, only whether the run terminates.

A second silent-drop channel has the same shape and the same bound. The emitted testbenches give the compute node's link to the network interface 16 credits against an 8-entry injection FIFO, so a sufficiently long injection burst would overflow the FIFO and the packet would be discarded with no flag. The presence gate bounds the damage exactly as above: a lost SEND leaves the

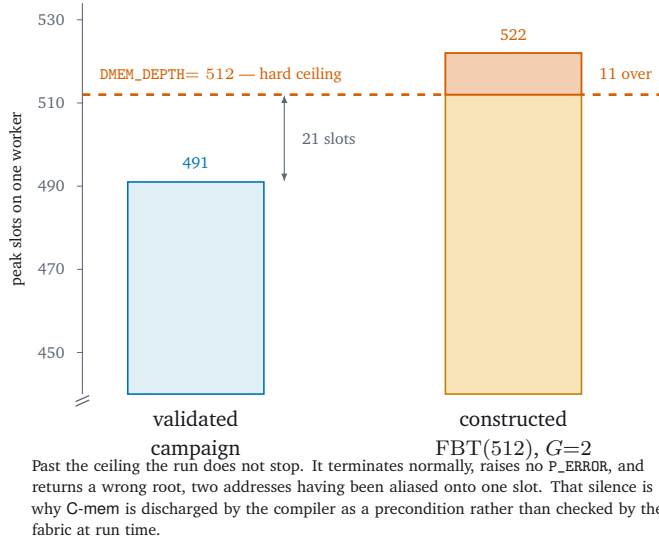


Figure 9. Peak DMEM occupancy against the hard depth. The campaign came within 21 slots of a ceiling that does not announce itself: past it the run terminates normally and returns a wrong root.

destination's bit clear, the dependent COMPUTE stalls, the watchdog fires. A drop can cost termination and never correctness, which is why no passing run can contain one.

4.7 Resource Envelope

Router cost is dominated by radix, and radix is a property of the topology rather than a constant of the design: the live parameter is `PORTS_PER_RTR`, emitted per instance and read back out of every generated testbench rather than assumed. It spans 3–66 ports across the suite — linear and ring at 3, mesh and torus at $P = 5$, hypercube reaching $P = 11$ at $G = 1024$, fat-tree reaching $P = 66$ at $G = 4096$ (`maxp_mod_fat`, 4,192 routers) — and on every topology at every size the emitted adjacency table wires exactly that many ports. Because the crossbar term is $O(P^2)$, a single fixed-radix estimate is wrong by more than an order of magnitude at the top of that range, so we report the envelope as a function of P at a fixed route-table depth of $G = 4096$, counting routing table, ingress and egress FIFOs and crossbar as storage-equivalent bits and excluding link drivers, clocking, arbiter logic and I/O. A fat-tree router is $20\times$ a mesh router (750,976 bits against 37,728), 56% of it crossbar (418,176 bits); aggregated over 4096 routers that is ≈ 147 Mbit at $P = 5$ against ≈ 3002.3 Mbit for `maxp_mod_fat` — the regime in which a flat crossbar stops being the right structure and a hierarchical or input-queued switch is the standard answer. We report the flat figure because it is what we simulated.

Per-PE memory is topology-independent but only half fixed: DMEM is 512×64 on every instance, while IMEM is elaborated to the next power of two above

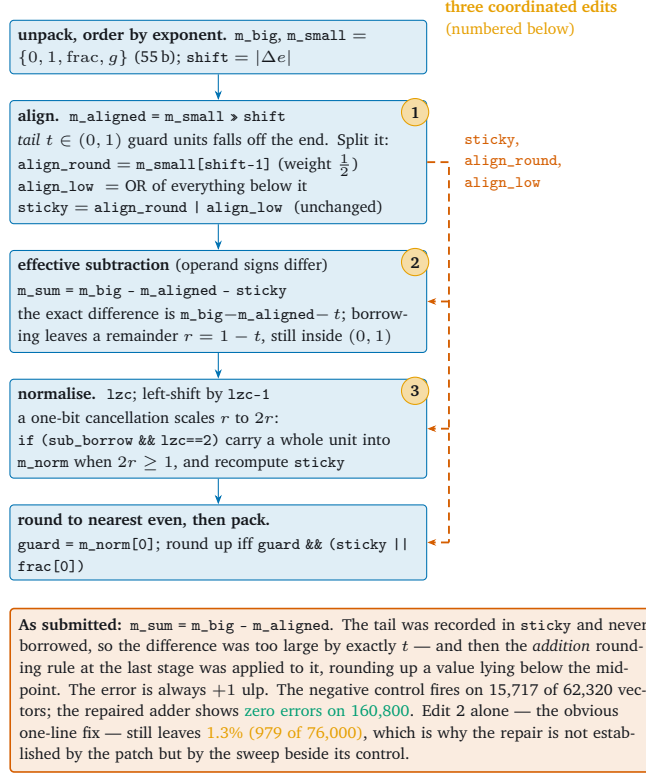


Figure 10. The effective-subtraction defect and its repair, on the adder datapath. The alignment tail was recorded in *sticky* but never borrowed from the difference, and the *addition* rounding rule was then applied to the result. The one thing to take from it is why the fix is three edits and not one: *sticky* threads every stage, so borrowing the tail changes what the renormalising shift and the final rounding decision mean. Edit 2 alone — the obvious one-line patch — still leaves 1.3% of effective subtractions wrong, which is why correctness here is established by the sweep beside its negative control and not by the patch.

that instance’s longest per-PE program, 64 to 2048 entries across the suite. We price the 512×64 case for both — 65,536 bits per PE, ≈ 256 Mbit over $G = 4096$ — as a design target, not as a measurement of what every run instantiated. All of these are RTL-level counts from a prototype whose ALU is combinational: we have run no synthesis and report no $F_{\max}$, slack, utilisation or power anywhere in this paper.

4.8 The Adder, and a Defect Only an External Standard Found

The adder we first submitted did not implement round-to-nearest-even on effective subtraction, and how it escaped is more useful than the patch. Fig. 10 gives the mechanism on the datapath. Scored against the host’s correctly-rounded addition the defect fires on 46.8%–50.4% of opposite-signed pairs whose exponents differ by 5 to 52 binades, and on *every* such pair at a difference of exactly 53.

The repair takes the three coordinated edits marked in Fig. 10. Stopping after the borrow, the obvious one-line fix, leaves a -1 ulp residual on 1.3% of effective subtractions (979 of 76,000 in a three-variant sweep), concentrated at exponent separations of 2–10 binades. A partial repair here looks plausible and is wrong, which is why what establishes correctness is the verification and not the patch. On a two-directional binade sweep plus a uniform random sample the repaired adder agrees with correctly-rounded IEEE 754 addition on all 160,800 vectors (zero errors) and the repaired software oracle agrees with it bit for bit on the same set (zero mismatches); over the wider host-FPU sweep the figure is zero of 4,644,000. The negative control fires: the as-submitted pair fails the same test on 15,717 of 62,320 vectors.

No number in this paper moves, and we checked that rather than argued it: replaying the emitted reduction order under the as-submitted and the repaired oracle over 53 worker counts from 2 to 16,384 moves zero roots and encounters zero mixed-sign additions, while the guard’s own injected-adder control detects two of two. The reason it moves nothing is the reason the campaign could never have found it, and that is L13’s subject (§11) rather than an erratum.

4.9 What the Fabric Does Not Do

Head-of-line blocking. The router is store-and-forward with a single FIFO per VC; under tree-reduction traffic the root-incident links queue up to $\lceil \log_2 G \rceil$ merge waves, and absolute cycle counts slew at $G \geq 1024$ on ring and torus because of it. Wormhole forwarding is a planned extension and does not touch the ABI.

Single clock domain. A multi-chiplet deployment at $G = 4096$ would need asynchronous FIFOs on inter-chiplet links, changing credit round-trip latencies but not the bit result.

Validated at one point of a parameterised design, and the unused halves are where the defects are. The fabric instantiates two virtual channels, two compute-node links and $k/2$ fat-tree core switches; the compiler uses one of each. The credit backlog register above can only overflow on a two-VC read; the network interface’s injection race can only fire on the second link; and for $k \geq 4$ the fat-tree emitter assigns pod-to-core ports above the declared radix, which the fabric discards, so the elaborated network uses a single core switch. Every emitted route is valid on that network and the surplus cores carry no traffic — but the analytic lower bound Ω (§6) min-cuts the intended edge set, so the fat-tree Ω ratios we report are computed against a *more* capable network than the one simulated. That is an understatement of our own performance, not a flattery, and we state it as one caveat on the RTL rather than as three separate errata.

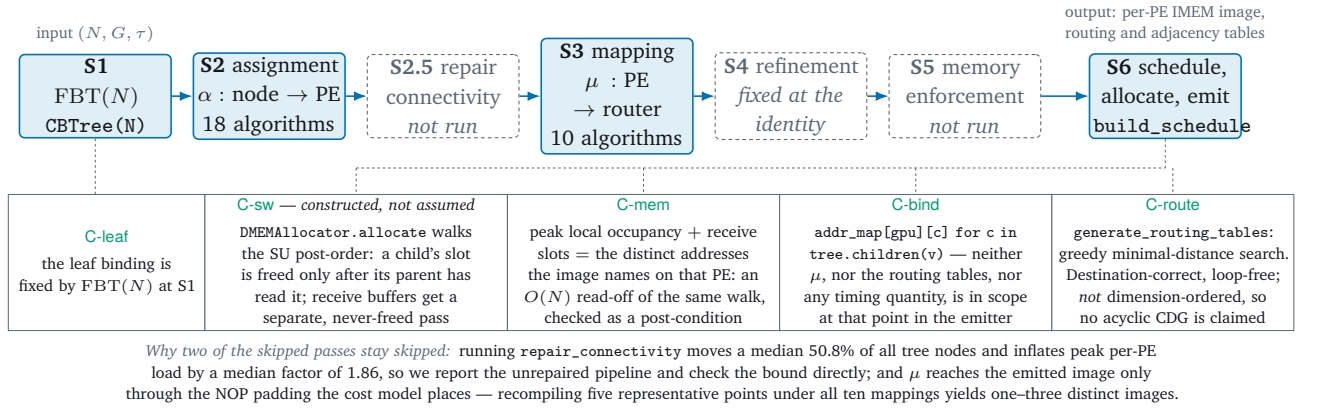


Figure 11. The compiler pipeline and where each obligation is discharged. Stages the campaign actually exercised are drawn solid; the assignment stage carries the whole decision.

Soft-enforced single-writer. C-sw is discharged by a compile-time post-condition over the emitted image, not by a hardware refusal of a second write to an already-set slot. A one-line RTL extension — raise P_ERROR on a scoreboard transition from 1 to 1 — would harden it into a fabric-side guarantee.

5 The Compiler

SPRS compiles $(N, G, \text{topology})$ into per-PE IMEM images plus routing and adjacency tables. Table 3 lists its passes and, for each, whether it produced the results reported here: five did, and three — connectivity repair (S2.5), memory enforcement (S5) and refinement (S4, twelve base methods chainable to 122 configurations) — did not. What follows describes the pipeline that ran.

One pass is load-bearing. The DMEM allocator walks the Sethi–Ullman-refined post-order. It reserves address 0 on every PE; at each node v on PE $p = \alpha(v)$ it returns to p 's free list the slot of every child of v already resident on p , then gives v the head of that list or else a fresh address. A final pass gives every cross-PE edge a persistent receive-buffer slot on the parent's PE, above that PE's local high-water mark, never freed. Three properties of that walk *construct* C-sw rather than *assume* it: FBT(N) is a tree, so every node has exactly one consumer; post-order visits children before the parent, so a local slot is freed only after its consumer has read it; and receive slots live in a separate never-freed pass, because a scoreboard-gated COMPUTE needs its address stable. An allocator working in pre-order, or freeing before the consumer ran, would break the discipline. The same walk yields C-mem free: peak local occupancy plus receive slots is exactly the number of distinct addresses the image names

Table 3. The compiler’s passes, and which of them produced the results in this paper. five ran; three did not, and we describe the pipeline that ran rather than the one the released tool can express. The 18×10 (S2, S3) face is enumerated exhaustively on every instance (§6); S4 is the axis the campaign does not cover (§11).

Pass	Alt. Status in the reported campaign
S0 lower bound Ω	– a reported metric, not a construction step (§6)
S1 FBT(N) instantiation	1 ran; fixes the binding of Defs. 4–5
S2 assignment α	18 ran; all 18 enumerated on every instance
S2.5 connectivity repair	1 never invoked; not load-bearing (below)
S3 mapping μ	10 ran; all 10 enumerated on every instance
S4 refinement r	122 fixed at the identity; no reported result varies it
S5 memory enforcement	1 never invoked; C-mem discharged as a post-condition
S6 emission, NOP padding	1 ran; constructs C-sw, emits the routing tables

on p , so the capacity obligation is an $O(N)$ read-off of a quantity the allocator already maintains.

The repair pass that looks load-bearing is not. Twelve of the eighteen assignment algorithms can hand a PE a node set that is not a connected subtree, and 2,061 of the 2,822 validated configurations are of that kind. Operand binding is unaffected (§3.3); what disconnection costs is DMEM, and it does not cost enough to matter: over 681 configurations peak per-PE occupancy has median 10 slots on the 244 connected partitions and median 25 (max 491) on the 437 disconnected ones, against the 512-entry budget. Running `repair_connectivity` relieves that pressure and destroys the load balance the assignment search exists to produce: it moves a median 50.8% of all tree nodes, inflates peak per-PE node count by a median factor of 1.86 and up to $304\times$, and leaves 82 of the 437 with more than 512 instructions on a single PE against exactly one before repair — and in 33 of them it *raises* peak DMEM. Connectedness is a correlate of memory pressure, not a guarantee of it, so we report the unrepaired pipeline and check the bound directly.

Mapping barely reaches the image. SEND carries the logical worker id and IMEM is loaded by logical index, so μ reaches the emitted image only through the hop counts the cost model uses to place NOPs: recompiling five representative points under all ten mappings yields one–three distinct images, differing only in NOP padding. Invariance across S3 is therefore close to tautological, and the

evidential weight of the campaign sits on the assignment axis and on reshape across $(G, \text{topology})$.

ABI verification. C-sw, C-bind and C-leaf are post-conditions on the emitted IMEM, checkable without executing anything. The campaign path did not run them in the compile loop, so we discharged them afterwards over the released images — 1,010,895 COMPUTE and 731,982 SEND instructions across 681 configurations, zero violations, no out-of-range address, no receive buffer with two writers. We report it as a static audit of the artifact rather than as a compiler feature, because that is what was run.

Schedule construction, and the one constant taken from the RTL. `build_schedule` walks the same post-order maintaining three per-PE clocks: when the PE is next free, when its GCI is free to issue the next GENERATE, and when each node’s value is available. A GENERATE on PE g cannot start before the later of the first two, and its data is ready $4 + 2$ cycles later — the state-machine count from `gci_start` to `gci_done`, plus one cycle for the pulse to propagate and one for the DMEM commit. That constant is read from the RTL rather than fitted, and the same $6\lceil N/G \rceil$ it induces is the GCI term of Ω (§6) and the dominant cost term at high N/G (§7.4). A cross-PE child emits a SEND on the child’s PE and the parent’s COMPUTE waits on the scoreboard, not on the cost model’s estimate of arrival: mispredicting arrival time costs cycles and cannot cost correctness. This is where the separation of concerns pays — the cost model is free to be wrong, which is what lets the campaign of §6 vary it freely without endangering the bit result.

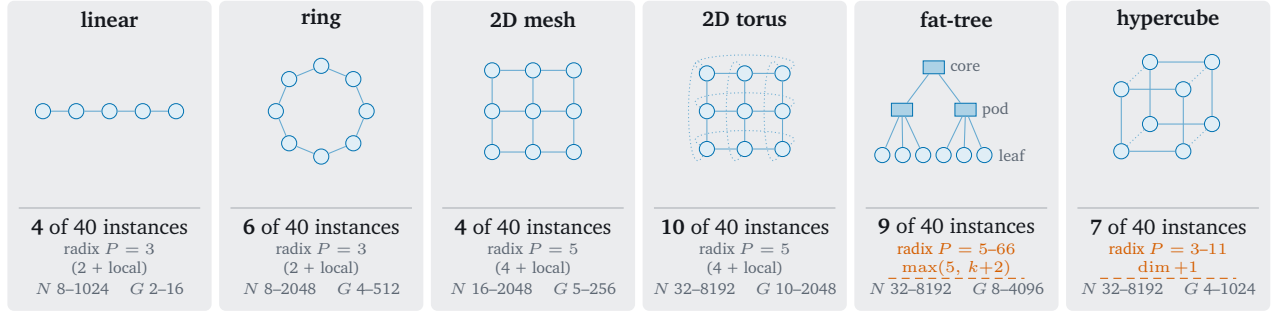
The line that realises C-bind is the operand lookup, `child_addrs = [addr_map[gpu][c] for c in tree.children(v)]`, where `tree.children(v)` returns `[left(v), right(v)]` by construction of $\text{FBT}(N)$, `addr_map` is the allocator’s output and `gpu` is $\alpha(v)$. Neither μ , nor the routing tables, nor any timing quantity is in scope at that point in the emitter, which is the code-level content of Proposition 1.

6 Methodology

Instance suite. 40 instances span $N \in \{8, 10, 16, 32, 50, 100, 128, 200, 500, 512, 1024, 1500, 2048, 3000, 4096, 8192\}$, $G \in \{2, \dots, 4096\}$, and the six topologies. Multiple $(G, \text{topology})$ points at the same N are what exercises the invariance claim. Leaf values are

$$L[i] = (i + 1) \cdot 1000.0 + ((i \cdot 37 + 13) \bmod 100) / 100 + 0.007, \quad (5)$$

chosen so every leaf carries a non-trivial fractional part: each FADD exercises the guard, round and sticky bits rather than reducing to an exact power-of-two addition, and magnitudes span 10^3 to 8×10^6 so partial sums accumulate over



One `noc_system` module realises all six: the adjacency table maps (router, port) to (target router, target port) and the routing table maps (router, destination) to an output port, both written before `program_start`, so a family is a Python classmethod and no Verilog change. Radix is what the shape costs. Linear and ring hold at $P = 3$ and mesh and torus at $P = 5$ whatever G is, while hypercube reaches $P = 11$ at $G=1024$ and fat-tree $P = 66$ at $G=4096$; because the crossbar is $O(P^2)$ a fat-tree router is $20\times$ a mesh router (750,976 against 37,728 storage-equivalent bits), which is why one fixed-radix estimate cannot price the suite.

What does not vary with the shape is the answer. Across all six families the target is $R_{\text{canonical}}(N, L)$, a function of N alone: over 2,822 settled hardware simulations no configuration produced a finite root differing from it.

Figure 12. The six topologies the instance suite spans, with how many instances sit on each and the router radix that follows. Measured radix spans 3 to 66, and a crossbar is $O(P^2)$.

a wide exponent range. We do not claim reordering *always* moves the result — it does not, which is what the mutation protocol below must work around. What the distribution buys is that a fabric/oracle mismatch is a determinism violation rather than an artefact of degenerate inputs.

Campaign protocol. The per-instance configuration space is $18 \times 10 \times 122 = 21,960$ (α, μ, r) triples; simulating it exhaustively would be $\approx 880,000$ xsim runs. We fix refinement at the identity and enumerate the remaining 18×10 face *exhaustively* on every instance — 180 triples per instance, 0.82% of the space, 7,200 compilations of which 6,777 emit an image. Distinct (α, μ) pairs frequently produce identical images up to PE relabelling; a SHA-256 hash over the emitted testbench collapses the 6,777 to 2,920 distinct ones. That is not merely a compute saving: identical images *tautologically* produce identical bit results, so dedup is what makes each remaining run informative. All 2,920 were queued for xsim bar maxp_mod_fat’s 76 (§11, L0), leaving 2,844 on the 39 hardware-validated instances — no shortlist of any kind stands between the compiler and the invariance evidence. A top- K -by-makespan shortlist would have missed a median 20% of the true top- K by measured cycles at $K = 15$, which is why we did not use one.

Mutation protocol. Two classes are injected per instance. *Address substitution* repoints `addr_a` of one randomly chosen COMPUTE at a different DMEM address. *Leaf corruption* perturbs one leaf by the smallest single-bit flip that provably moves the canonical root. The escalation in the second is necessary and is the same phenomenon the paper is about: flipping the mantissa LSB frequently falls below the round bit of the running sum, RNE discards it, and a fabric reporting PASS is correct to do so. We raise the flipped bit until the oracle confirms

the root has moved, so every counted mutant tests detection rather than rounding. We deliberately do not use operand swap: fadd_{rtl} is commutative at the bit level under FTZ+RNE, so it would be bit-identical and would inflate the null-detection rate.

Metrics. $\text{hw_cycles} = \max_p \text{perf_total_cycles}[p]$ from per-PE counters is primary; the cost model’s makespan is a diagnostic only. We also compute a composite lower bound Ω on any schedule on this fabric,

$$\Omega = \max(\text{lb}_{\text{span}}, \text{lb}_{\text{comm}}, \text{lb}_{\text{gather}}, \text{lb}_{\text{bis}}, \min_{K \in [K_{\min}, K_{\max}]} \max(\text{lb}_{\text{gen}}(K), \text{lb}_{\text{instr}}(K))), \quad (6)$$

where K is the number of PEs a schedule populates, $K_{\max} = \min(G, 2N - 1)$. The terms are the zero-communication critical path, GCI serialisation, instruction throughput, a cross-PE hop floor, the root PE’s gather path, and link serialisation at the bisection. Two of the six shrink as work spreads over more PEs, so a bound valid against *every* schedule must take the most favourable K ; a maximum there would assert a bound some legal schedule beats. Because Ω bounds non-deterministic schedules too, it is not a cost-of-determinism denominator but an absolute-performance diagnostic. Summary statistics are medians with observed ranges rather than interval estimates: the per-instance samples are not draws from a population, they enumerate a deterministic configuration space, so a resampling interval would describe our sampling of that space rather than any underlying uncertainty.

Campaign accounting. The campaign cost 1,003.3 core-hours: 821.5 on passing runs, 122.9 on runs that did not settle including retries, and 58.9 on the one instance that never settled at all (§11, L0). Feasibility is not uniform across the suite: DMEM is fixed at 512 entries per PE but IMEM is elaborated per testbench, and 278 of the 7,200 compilations exceed the 512-instruction reference budget and are flagged E_IMEM_OF. They were emitted and simulated anyway and 113 of them are among the validated configurations. They cluster at both ends of the N/G range — high N/G piles leaves onto too few PEs, while at $N/G = 2$ a PE with few local nodes has many remote children to land — and instruction-memory depth cannot change a root, so the 512-entry budget is the deployment target priced in §4.7 rather than the depth every simulation used.

The baseline we did not build. The natural metric is the same-fabric overhead η against the best non-deterministic reducer on the identical substrate, obtained by relaxing C-bind. **We did not build it and report no measured η** , and we decline to substitute an analytical one. We did build that model — the same cost constants that produce Ω , applied to a ring and to recursive doubling — and it returns $\hat{\eta} \approx 1$ on every instance. Where N and G are powers of two that is no modelling artefact, and the reason is structural. The ISA’s COMPUTE is two-input, so every reducer over N leaves has depth at least $\lceil \log_2 N \rceil$, which is

exactly the depth of $\text{FBT}(N)$: the tree-depth term is ≤ 0 on all 39 validated instances and SPRS is never deeper. At power-of-two N and G , recursive doubling over contiguous blocks is DAG-isomorphic to the ABI schedule — verified by direct construction on 88 of 88 such pairs — so the two reducers are the same computation, and $\eta = 1$ is a theorem rather than a measurement we failed to take. On the 31 power-of-two instances there is nothing a baseline could reveal; on the other 8 it fails, and a model blind to which operands pair is not evidence there.

That argument is about *depth*, and depth is what binds in the regime the fabric evaluates: one FP64 value per leaf — $D = 1$ in the sense of §9 — where a reducer’s cost is its dependence chain and not the bytes it moves. At large message size a bandwidth-optimal collective is bounded by the other term, and there a pinned tree does cost something. We do not estimate that cost either. §8 measures the same $\text{FBT}(N)$ discipline against stock `ncc1AllReduce` on production hardware, and finds at $G=2$ an algorithmic factor of about two in bytes on the wire that no engineering removes. The two regimes are priced by different instruments — a construction argument and a measurement on real collectives — and neither result is offered as evidence for the other.

A baseline on the remaining 8 would say something, and neither natural one — the physical worker ring, or recursive doubling at non-power-of-two scale — is a small change to build: relaxing C-bind means re-deriving the allocator’s single-writer discipline and the scoreboard gating around a schedule with no canonical operand pairing, and an unoptimised baseline would measure our relative implementation effort rather than the ABI. We flag that residual as the principal open item in the evaluation (§11, L1b) and price the ABI structurally instead (§7.4).

7 Evaluation

Vivado xsim 2025.2 on Linux; ten SystemVerilog sources (3,738 lines); a 72-thread Intel Xeon w9-3475X with 503 GB of RAM; RNG seeds pinned at 42; per-simulation wall-clock cap 28,800 s; the supplemental material lists the artefacts and states exactly what each per-run verdict is. Every cycle figure comes from a single campaign of 2,822 passing runs consuming 821.5 core-hours at a median of 99 s per testbench, the five largest instances accounting for 90.0% of that and the longest single run taking 5.51 h. An earlier campaign over the same testbenches is used for no reported number: it recorded PE 0’s counter rather than the maximum across PEs, which understates end-to-end latency whenever the root lands elsewhere. That defect moved 50.5% of cycle counts and changed the *minimising configuration set* on 38 of 39 instances,

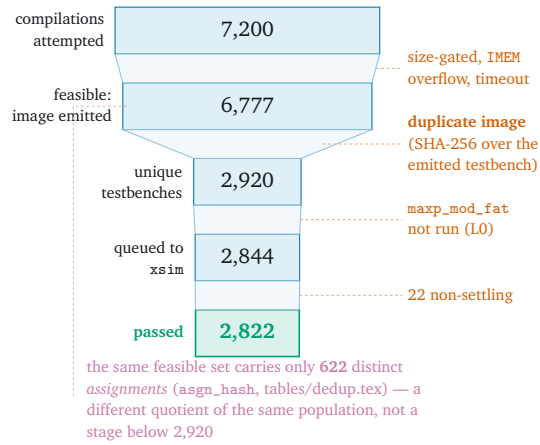


Figure 13. The campaign funnel, from compilations attempted to passing hardware runs.

leaving it disjoint from the corrected set on 32, so the measurements were repeated in full. Hardware validation covers 39 of the 40 instances (§11, LO).

7.1 The Premise, Measured

The ABI is worth its cost only if reshaping a production collective really does move the bits. On a single node with two NVIDIA RTX PRO 6000 Blackwell GPUs over PCIe (no NVLink) and 72 CPU cores we reduced identical input *bytes* at fourteen rank counts from 1 to 64 under PyTorch 2.11.0’s distributed collectives [?], holding data, seed and code fixed so that only the shape of the job changed. Leaf *i*’s value is a pure function of *i*, so reshaping alters who holds which leaf and in what order the partial sums combine, and nothing else.

The fourteen configurations returned fourteen distinct 64-bit results in every one of twelve (leaf count, precision, value-profile) cases; in the six fp64 cases, for which the single-process canonical reduction was also computed, none of the fourteen matched it. Spreads reach 5,120 ulp and a maximum relative spread of 8.1×10^{-13} ; at least 1,023 of the 1,024 elements of the result vector move in every case; pooled over every configuration swept, 78.6% of fp64 configuration pairs and 90.0% of fp32 pairs disagree. Repeated runs at any *fixed* configuration were bitwise identical five times out of five, with no disagreement between ranks inside a run, and an independent re-run under a different bookkeeping harness reproduced 168 of 168 common cells bit for bit. Reshape moves the answer; noise does not.

Three scope conditions belong with the measurement rather than under it. The rank-count sweep above $G=2$ runs on the CPU backend, because NCCL requires one rank per device and this node carries two GPUs. At $G=2$ the NCCL algorithm-by-protocol axis (seven configurations) and the channel-count axis (three) each returned a single root; that is structural rather than an empirical

null — at two ranks the cross-rank reduction is one addition per element and no schedule can reassociate it — and we do not generalise it to larger G . At that same partition the CPU and GPU roots differ in all twelve cases, so device placement moves the bits as well as partitioning does. This measures reduction order on one node; it is not a benchmark of any library and no timing claim is drawn from it.

7.2 Cross-Topology Bitwise Invariance

We state the primary empirical result in the narrowest form the campaign supports. Across the 2,844 distinct ABI-compliant program images obtained by compiling the complete 18×10 grid on the 39 hardware-validated instances — sixteen leaf counts $N \in [8, 8192]$, requested worker counts $G \in [2, 2048]$, six direct-network topologies — 2,822 simulations terminated, and in every one the root PE’s final COMPUTE result was bit-identical to the canonical root $R_{\text{canonical}}(N, L)$ the software oracle predicts for that N . The remaining 22 did not settle; no configuration produced a finite root differing from $R_{\text{canonical}}(N, L)$.

Two readings of that sentence are available and the weaker one is the one the claim needs. Each emitted testbench carries the canonical root as a compile-time EXPECTED constant, and that constant is a function of N and the opcode alone: it names no G , no topology, and no assignment, mapping or schedule. A PASS therefore certifies that the run’s root equals a value fixed by N — so every settled run at a given leaf count produced the same 64-bit root as every other settled run at that leaf count, across all its $(G, \text{topology})$ points and all its compiler triples. That is Definition 1 verbatim, and it does not depend on the oracle being an independent model of IEEE-754 addition, which §11 (L13) records it is not. What does depend on the oracle is the stronger reading, that the shared root is the *correct* $R_{\text{canonical}}(N, L)$. We report both and keep them apart.

Two scope conditions belong inside that sentence rather than in a footnote. The campaign evaluates one leaf vector per N and the identity refinement throughout; and eight of the sixteen leaf counts are represented by a single requested $(G, \text{topology})$ point, so at those N invariance is witnessed across assignment, mapping and schedule but not across reshape. 2,403 of the 2,822 configurations lie at leaf counts carrying two or more points, and that is where the reshape evidence sits.

Where reshape is exercised it is not marginal. At $N=128$ the same root survives six $(G, \text{topology})$ points from $G=4$ on a linear chain to $G=128$ on a torus — a $32\times$ change in worker count across five distinct topologies — and at $N=512$ and $N=1024$ it survives six and five points. The 2,844 images are moreover distinct

Table 4. Cross-topology bitwise invariance. Each row aggregates all HW-validated SPRS configurations at the given N over all same- N (G , topology) pairs and all compiler triples. “Divergent” is by Claim 1 expected to be zero. The two mutation columns count injected mutants the harness *flagged*; as discussed below, a flag is a value mismatch for leaf corruption but usually a fail-stop for address substitution, and only the former bears on oracle non-vacuity.

N	compiled (G , topo)	HW-val. (G , topo)	HW configs (passing)	64-bit root result	Div.	Addr-sub detected	Leaf-corr. detected
8	2	2	28	0x40E1948E978D4FDF	0	6/6	6/6
10	1	1	17	0x40EADBA0A3D70A3E	0	3/3	3/3
16	1	1	21	0x41009A44BC6A7EFA	0	3/3	3/3
32	3	3	102	0x41201D1FCED91687	0	6/6	6/6
50	1	1	25	0x413374911999999A	0	3/3	3/3
100	1	1	33	0x415343B08CCCCCD	0	3/3	3/3
128	6	6	371	0x415F7E8FF9581062	0	6/6	6/6
200	1	1	44	0x41732B4046666667	0	3/3	3/3
500	1	1	98	0x419DDCAB2C000000	0	3/3	3/3
512	6	6	456	0x419F4FA404418938	0	6/6	6/6
1024	5	5	425	0x41BF47D2026872B1	0	2/2	2/2
1500	1	1	104	0x41D0C665F8400000	0	2/2	2/2
2048	4	4	437	0x41DF43E900FBE76D	0	2/2	2/2
3000	1	1	77	0x41F0C4F764200000	0	2/2	2/2
4096	3	3	331	0x41FF41F48085A1CB	0	2/2	2/2
8192	3	2	253	0x421F40FA40427EFA	0	1/1	2/2

“Compiled” counts the distinct (G , topology) points at that N for which an ABI-compliant image was produced; “HW-val.” counts those carried through RTL simulation. The single shortfall is $N=8192$, where `maxp_mod_fat` ($G=4096$, fat-tree) could not be simulated; the reason is not a wall-clock budget (§11, L0).

programs, not repeated runs of one program: the campaign deduplicates by SHA-256 over the emitted testbench and simulates one representative per hash class, so two images in the count differ structurally, not merely in the triple that produced them. Invariance holds simultaneously under reshape and under complete replacement of the assignment and mapping algorithms.

The campaign has since been re-executed from source: every passing configuration at $G \leq 64$ — 1,326 runs over twenty-five suite instances spanning all six topologies — was recompiled and re-simulated under both route-table loading paths (§11, L0b), and on the 1,306 points that completed under both, the two paths agree with each other and with the oracle on the same 64-bit value, one distinct root per instance. The released results file records verdicts and cycle counts but not root words, and the RTL re-simulated is the current tree rather than the exact one that produced that file, so this is a re-execution that self-agrees and re-derives the oracle’s prediction, not a bitwise diff against the originally released values. We state it in those terms.

	fail-stop P_ERROR, watchdog	value oracle root $\neq$ $R_{\text{canonical}}$	no verdict recorded
address substitution 54 runs, C-bind	at least 42 no comparison ran	at most eleven reached a live slot	1 maxp_dense_hc
leaf corruption 54 runs, C-leaf	0	54 all terminate normally	0
of 108 runs	43 Property 2 outcome (b)	61–65 the non-vacuity evidence	1 unclassifiable

A zero-divergence column means nothing unless the oracle can report a non-zero one. Only the middle column can: non-vacuity rests on 61–65 of the 107 verdicts, not on all 107. The left column is not waste — it is Property 2's halt observed under injected fault. Hatched cells are bounds, not counts: the route was recovered by static reconstruction, and one run returned no verdict at all.

Figure 14. Mutation testing: what each injected fault class does, and by which route it is caught. Reordering genuinely changes the root, which is what makes an agreeing run evidence rather than a tautology.

Table 4 reports the result, with the divergence column zero as Claim 1 predicts. Its rows are not equally informative, for the reason §3.1 measures: at the smallest leaf counts a large majority of alternative association orders reproduce the canonical root anyway, so agreement there is close to free and the weight of the evidence sits at the large- N rows.

Beyond the measured range. Invariance at untested G rests on the structural argument, not on induction: operand binding and scoreboard gating remain total functions of N and L as G grows, so adding PEs changes the *location* of FADD operations, not the operand pairs they receive. Adding a PE does introduce new routing paths, VC contention and credit interactions that may move cycle counts. What carries is the bit result; absolute cycle counts at large G are simulation artefacts, not silicon measurements, and we make no absolute competitiveness claim. Indeed we make no silicon claim at all: we ship Artix-7 bring-up wrappers for $G \in \{2, 4\}$, but synthesising them needs a Vivado VIO IP core and board constraints we do not redistribute, and we hold no board logs. That is a reporting gap rather than a soundness gap — invariance is a statement about bit values, which xsim resolves exactly.

7.3 The Oracle Is Not Vacuous, and Where It Stops

A zero-divergence column means nothing unless the oracle can report a non-zero one. Reporting the injected-fault campaign requires more care than a single rate. The harness retains one boolean per mutant, and the testbench prints [FAIL] both on a value mismatch and from its !all_done branch on any run that fails to complete; the two are not distinguishable in the recorded output. Reconstructing the mutations against regenerated images separates them. Of the 53 address-substitution verdicts, in at least 42 the value com-

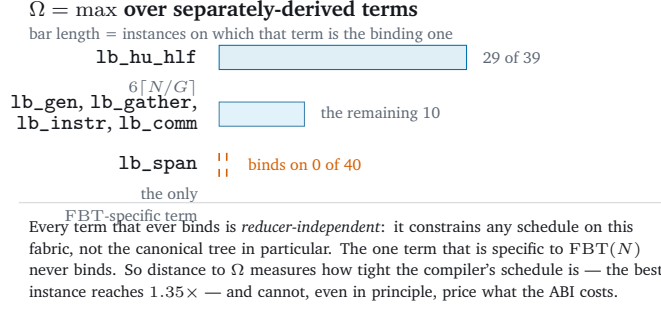


Figure 15. How Ω is assembled, and which constituent binds. Every term that ever binds is reducer-independent; the one FBT-specific term binds on none, which is why distance to Ω cannot price the ABI.

parison never ran: the substituted address is one no agent ever writes — the harness draws a uniformly random address rather than another node’s — so the dependent COMPUTE stalls on the presence gate and the flag is a fail-stop, not a detection. Oracle non-vacuity therefore rests on 61–65 of the 107 verdicts — every leaf corruption, plus the address substitutions that happened to reach a live slot — and not on all 107, as we previously reported.

Read the other way round, the same fact is an asset. 43 of the 54 address substitutions fail-stopped instead of corrupting, and in every one we re-simulated the fabric raised `P_ERROR` with a recorded `error_pc` on every PE. That is Property 1’s halt outcome observed under injected fault rather than argued from the RTL, and the split falls exactly where the presence gate places the boundary: all 54 leaf-corruption mutants terminate normally with a wrong root — one of them one ulp from canonical — and are caught by the value comparison instead. Mutants were injected across all sixteen leaf-count classes, so no row of Table 4 rests on an oracle that was never exercised.

7.4 The Cost of the ABI

We report no measured η (§6); quoting a ratio against a reducer that does not exist would not be a measurement. But the ABI’s cost is a consequence of the contract, and the contract is small enough to price analytically. Four constraints separate SPRS from an unconstrained reducer:

1. *FBT depth, which is not a cost.* The ABI fixes $\text{FBT}(N)$, so the dependence chain is $\lceil \log_2 N \rceil$ deep regardless of G . Since COMPUTE is two-input no reducer here forms a partial sum of higher arity, so none is shallower: this term is ≤ 0 on all 39 instances (§6) and enters the account as a zero.
2. *No reassociation.* Fixing $(\text{left}(v), \text{right}(v))$ forbids regrouping operands by locality. An unconstrained reducer may sum PE-local values first and cross the network once; SPRS crosses wherever the tree says to.

3. *GCI serialisation.* Consecutive GENERATES on one PE are spaced by `gci_gap` = 6 cycles, so a PE holding $\lceil N/G \rceil$ leaves spends at least $6\lceil N/G \rceil$ cycles producing them. Unlike term 2 this is an artefact of *our* GCI, not of the ABI.
4. *Root-gather fan-in.* The root PE must receive from the other $G - 1$ PEs: at least $\lceil \log_2 G \rceil$ merge stages plus roughly half a network diameter. This binds any reducer, deterministic or not.

Term 2 alone is the price of reproducibility and cannot be engineered away: what separates SPRS from a free reducer is which operands pair at each level, not how many levels there are. Term 3 is ours to fix, term 4 nobody's.

Term 3, measured directly. The GCI floor $6\lceil N/G \rceil$ accounts for at most 30% of the champion makespan on every one of the 7 instances with $N/G \leq 2$, and for at least 43% on every one of the 14 instances with $N/G > 8$, rising to 64–74% on the 5 instances with $N/G \geq 32$ (Spearman $\rho = +0.77$, $n = 39$). The two groups do not overlap. Since $6\lceil N/G \rceil$ assumes perfectly balanced leaf assignment while any real assignment puts at least $\lceil N/G \rceil$ leaves on the busiest PE, these are *lower* bounds on the true share. Leaf-generation serialisation is thus the dominant term at high N/G : on this suite a multi-channel leaf interface is the highest-value modification available to the fabric, and it costs nothing in the ABI.

The obvious test of term 3 is circular, and we withdraw it. Banding cycles/Ω against N/G and reading a knee off the decline measures the denominator: Ω equals $6\lceil N/G \rceil$ on 29 of the 39 instances, the absolute distance $\text{cycles} - \Omega$ carries no N/G signal at all ($\rho = +0.09$, $p = 0.58$), and a one-parameter model with no mechanism in it, $\text{cycles} = \Omega + 96$, reproduces the band medians to within 0.13. Nothing is lost — the direct measurement above does not involve Ω — and the sign is the opposite of what such a reading suggests, since GCI serialisation binds *above* $N/G = 8$, not below it.

Distance to the lower bound. The one absolute statement available is how far the compiler lands from Ω . Because Ω bounds any schedule here, including a non-deterministic one, cycles/Ω upper-bounds what an ideal unconstrained reducer could have saved: weaker than η , but real and measured. The informative reading is additive rather than multiplicative. The compiler's absolute distance from the bound is near-constant — median 96 cycles, interquartile range [60, 147] — across a range of Ω from 13 to 384, and uncorrelated with N/G ($\rho = +0.09$, $p = 0.58$). The schedule carries a roughly fixed overhead over the analytic floor regardless of problem size, so relative efficiency improves monotonically with scale and reaches $1.35\times$ at the largest instance. No instance reaches Ω , which is expected: the bound is assembled from separately-derived constraints and is not claimed

Figure 16 is the per-instance picture behind Table 5, which is a descriptive

Table 5. Distance to the composite lower bound Ω by topology, over the 39 HW-validated instances. Ω bounds any schedule on this fabric, so these ratios upper-bound what an unconstrained reducer could have recovered. Bars share one axis. The one thing to take from it: the ordering tracks bisection width — linear and hypercube land closest to the bound, ring furthest — and no topology reaches it.

Topology	Inst.	Median cyc/ Ω	Best
linear	4	1.39 	1.35
hypercube	7	1.85 	1.55
torus	10	2.26 	1.49
mesh	4	2.58 	1.62
fat-tree	8	3.40 	1.83
ring	6	4.48 	2.33
all	39	2.34 	1.35

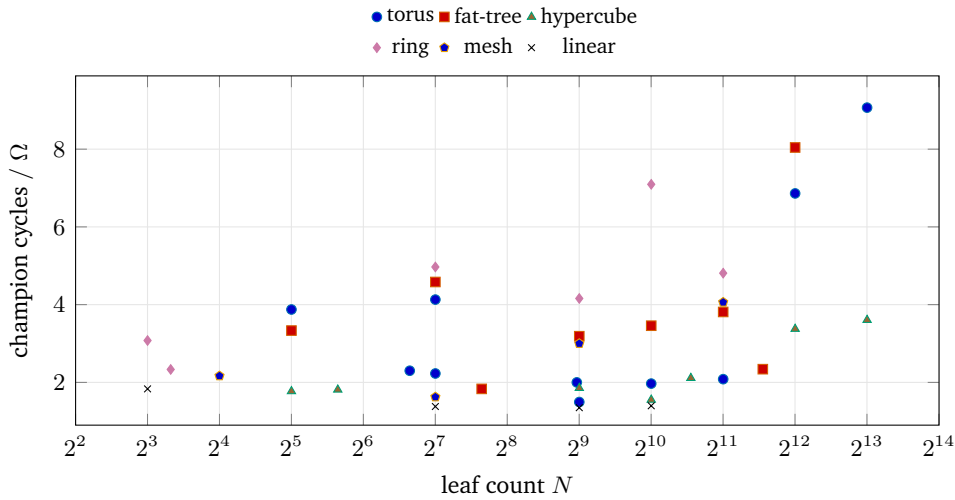


Figure 16. Distance to the composite lower bound Ω for every HW-validated instance, against leaf count. A value of 1.0 would mean the compiler achieved the bound exactly; Ω is assembled from separately-derived constraints and is not claimed tight, so the gap is part slack and part real. The informative reading is additive, not multiplicative — see the text.

per-topology summary and not a controlled contrast, and we draw no topology ordering from it: the six cells differ in median N/G by a factor of 24, the suite contains only two (N, G) points carrying more than one topology, and those two disagree in sign. Any ordering read off that column is substantially N/G in disguise.

7.5 Champion Equivalence Classes

Earlier versions of this paper reported a winner distribution — which assignment and which mapping algorithm produced the fastest image on each

Table 6. Held-out performance of the portfolio against its size k . Leave-one-out is the optimistic protocol; holding out a whole topology is the honest one, since instances sharing a topology are not independent. $k=4$ is the knee: $k=5$ is identical to four decimals. Intervals are 95% bootstrap CIs, $B=2000$, resampling the 39 instances.

k	leave-one-out		unseen topology		% optimal
	mean	95% CI	mean	95% CI	
1	1.0797	—	1.0797	—	41.0%
2	1.0305	[1.0171, 1.0454]	1.0708	[1.0314, 1.1245]	43.6%
3	1.0111	[1.0030, 1.0209]	1.0461	[1.0099, 1.0955]	82.1%
4	1.0025	[1.0000, 1.0076]	1.0391	[1.0025, 1.0885]	89.7%
5	1.0025	[1.0000, 1.0076]	1.0391	[1.0025, 1.0885]	89.7%
6	1.0000	—	1.0366	—	92.3%
7	1.0000	—	1.0366	—	92.3%
8	1.0000	—	1.0000	—	100.0%

instance. We withdraw it, and the reason is this paper’s own subject. The campaign deduplicates by program image before simulating, so one simulated row stands for a class of bit-identical programs and an instance’s champion is a *set*. Expanded back over the full 18×10 grid, the minimum measured cycle count is attained by more than one configuration on 38 of 39 instances, by a median of ten and by as many as 90; all ten mapping algorithms attain it on 27 of 39; and exactly one instance has a unique champion. Which member of a tie set the earlier table named was decided by a minimum over CSV rows in parallel-worker arrival order — by which simulation host finished first. No tie-break repairs that: on 38 of 39 instances the tied set contains programs that are bit-identical and cannot be separated by any measurement at all.

What replaces the ranking is stronger, and its data was already in the paper. Proposition 1 says the compiler’s variation space cannot reach the bit result. The campaign shows that same space collapsing structurally before it ever reaches the fabric: of the 7,200 compilations performed, 59.4% produce a program image that another configuration had already produced. The compiler variation space is thus not merely incapable of changing the answer — over most of its extent it does not even yield a different program. The campaign is, from the ABI’s point of view, 2,822 attempts to break Claim 1, every one of which failed; it is not a ranking of algorithms. What survives of the per-algorithm question is champion-class *membership*, which is invariant to the tie structure and which the supplemental catalogue reports for all eighteen assignment algorithms.

7.6 A Derived Selection Rule

The campaign was built to answer a practitioner’s question — given an instance, which of the 180 configurations should be run? — and champion-class

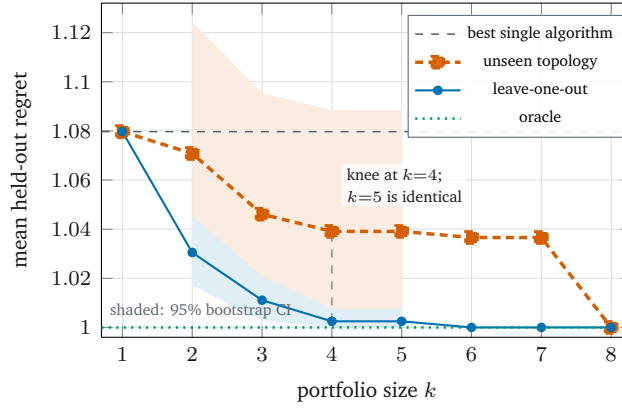


Figure 17. Held-out regret against portfolio size. The rule transfers to a topology it never saw, and conditioning the choice on topology is worse than ignoring it.

membership answers it in a form invariant to the tie structure above. For a recommended set we report *regret*: the measured cycle count of its best member divided by the instance’s measured optimum, so 1.000 means the set contains a champion.

The rule is a portfolio, not a regime map. Exhaustive search over subsets of the sixteen near-universally evaluable assignment algorithms, minimising mean measured regret and refitted independently in each of 39 leave-one-out folds, returned the *identical* subset in 39 of 39 folds at every size. At size four that subset is DSC, Lukes, Frederickson, ParSub, and on the held-out instance it attains the exact hardware-measured optimum in 38 of 39 folds (0.974), with mean out-of-sample regret 1.0025 and worst case 1.099. It evaluates 4 of the 180 configurations, a median $144.6\times$ reduction in compiler time (205.6 s to 2.78 s per instance). Table 7 gives it with the smaller budgets and, for completeness, the restricted-subset rows a regime map would use; the row to adopt is the unconditional default, for the reason the next paragraph gives. Size four is a knee rather than a budget we picked: a fifth configuration is identical to four decimal places under both protocols, and only at six does held-out regret reach 1.0000.

Conditioning on the instance makes it worse. This is the result we did not expect and the one that makes the portfolio finding non-obvious. Every decision tree we could fit over $(N, G, N/G, \text{topology})$ is beaten out of sample by the unconditional portfolio: the best reaches an exact-optimum rate of 0.564 (22 of 39) against 0.974, and its out-of-sample mean regret, 1.21–1.31, is worse even than the single-algorithm constant rule “always ParSub” (1.0797). A regime map over these axes is not merely unnecessary; it is actively harmful, because it fits partitions of a 39-instance suite whose cells hold two or three instances each. The simplest conditioner fails the same way: taking whichever algorithm was best on this instance’s topology in training attains the optimum on 21 of 39 held-out instances against 38, at mean regret 1.0488 against 1.0025. Topology

is the axis on which conditioning is most intuitive, and the axis on which it most clearly does not pay. Against the two pre-specified constant baselines the portfolio is significant on both confirmatory tests (McNemar $p = 0.0005$; Wilcoxon signed-rank on regret $p = 0.0003$; both survive Holm correction over the two comparisons).

It transfers to a topology the fit never saw. Leave-one-out holds out an instance but not its topology, and instances sharing a topology are not independent, so we also refit with an *entire topology* withheld and score each fold only on the topology it never saw. Across the six topologies the portfolio transfers better than conditioning generalises: mean out-of-sample regret 1.0391 (95% bootstrap interval [1.0025, 1.0885] over 2,000 resamples of the 39 instances), lower than the 1.0488 of a rule that conditions on topology and does see it in training. The exact measured optimum is still attained on 35 of 39 instances (89.7%), worst case 1.675. Withholding an entire leaf-count band instead leaves the leave-one-out figure unchanged (1.0025). We quote the interval and not the point estimate deliberately: with only six topologies the grouped estimate is genuinely uncertain at the top end, and a practitioner adopting the rule on an unseen topology should plan against 1.0885 rather than against the mean. The corresponding leave-one-out interval is [1.0000, 1.0076].

The mapping stage can be deleted. Assignment is the whole decision. Conditional on ParSub, the ten mapping algorithms give byte-identical cycle counts on all 39 instances. Fixing the mapping at Identity and choosing only the assignment attains the exact optimum on 38 of 39, at mean cost 0.046% and worst case 1.8%. We therefore recommend removing the mapping search from the pipeline rather than conditioning on it. Note what this does *not* say: Identity is not the best mapping — its apparent lead in the withdrawn winner table was a labelling artefact (§7.5) — it is indistinguishable from the others and the cheapest to compute. In the other direction the spread across assignment algorithms is three orders of magnitude larger: Six of the eighteen are in the champion class on *zero* of 39 instances and cost a median $2.3\times$ to $13.1\times$ in cycles, up to $51.5\times$ at worst. Greedy list scheduling is the block to avoid.

What the rule does not license. Nothing here survives Holm correction over the 36 exploratory association tests we ran between algorithm families and instance parameters; two survive Benjamini–Hochberg, and we report the portfolio rather than any of them precisely because the portfolio needs no such association to hold. The suite could not support a conditional rule in any case — N and G are confounded by construction and most ($\text{size} \times \text{topology}$) cells hold two instances or fewer (§11, L5) — so we derive no cell-level recommendation. The rule is untested at $G = 4096$, the one design point without a hardware result. And it is a retrospective statement about measured cycle counts, not a claim that the compiler can identify the winner from its own cost model: every number is a measured cycle count on this pipeline for tree-structured reduction

Table 7. Derived algorithm-selection rule. For each regime, the S2 assignment algorithms a practitioner should run; the S3 mapping may be fixed to Identity throughout (Sec. 7.6). “Opt.” counts instances on which the recommended set attains the hardware-measured minimum cycle count exactly.

Regime	n	Run these S2 algorithms	Opt.	Mean regret	Worst regret
Default (any N , G , topology)	39	DSC, Lukes, Frederickson, ParSub	38/39	1.003	1.099
restricted to non-ring	33	Lukes, Frederickson, ParSub	32/33	1.003	1.099
restricted to ring [†]	6	DSC, ParSub	4/6	1.019	1.063
Budget $k=1$: $N \geq 2048$	10	ParSub	7/10	1.094	1.607
Budget $k=1$: $N < 2048$, non-ring	24	Lukes	17/24	1.076	1.853
Budget $k=1$: $N < 2048$, ring [†]	5	DSC	2/5	1.092	1.300
Budget $k=2$ (any instance)	39	DSC, ParSub	19/39	1.030	1.220
Budget $k=3$ (any instance)	39	DSC, Lukes, ParSub	34/39	1.011	1.118

[†]Rests on six (resp. five) ring instances of which three are positive; reported for completeness, not recommended as a standalone rule. No (N -bucket $\times$ topology) cell in the suite holds more than three instances, so no cell-level rule is derivable.

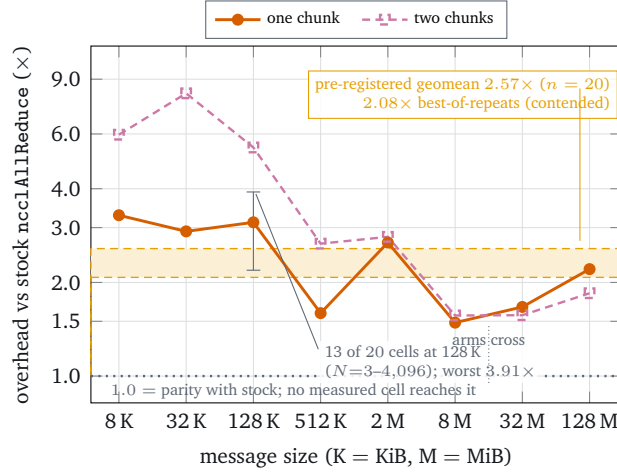


Figure 18. Measured overhead of the pinned-FBT export against stock `ncclAllReduce`. The headline is pre-registered and unfavourable; the point estimate is provisional, having been measured under GPU contention.

programs, and none of it transfers to another compiler, another accelerator, or to MPI or NCCL collectives.

8 Exporting the ABI to Production Collectives

Can the discipline be adopted without the SPRS fabric? It can, in software: replace a production collective’s dynamic G -dependent tree with a library that enforces the $\text{FBT}(N)$ post-order schedule whatever G and the topology are. We call this *pinned-FBT*. Two things must be said first, because we got the first

wrong and the second is not ours.

The obvious construction is incorrect. It partitioned canonical leaf indices into contiguous ranges, $\text{rank}(i) = \lfloor iG/N \rfloor$, and asserted that each rank then reduces exactly the subtree rooted at the lowest common ancestor of its leaves. Swept over all 44,850 pairs with $2 \leq N \leq 300$ and $2 \leq G \leq N$ that holds on 1,217 of them (2.71%); the smallest counterexample is $N = 3, G = 2$, where rank 0 owns leaves $\{0, 1\}$ whose lowest common ancestor is the whole tree’s root; and it fails on 8 of the 40 instances evaluated here. Nor is it merely a description error: evaluated literally in FP64 on our own leaf vector (5), the composed construction returns a root differing from $R_{\text{canonical}}(N, L)$ on 15,630 of those pairs, on 45.2% of the 1,546 pairs our own leaf counts admit, and on two of our own instances — $N = 10, G = 4$ by 2 ulp and $N = 200, G = 12$ by 1 ulp. The discrepancy is 1–3 ulp throughout, small in magnitude and total failure of a bitwise claim. A second, independent error survives any repair of the first: an all-reduce over sub-roots “consistent with $\text{FBT}(N)$ ’s top $\log_2 G$ levels” is $\text{FBT}(G)$ only when N and G are both powers of two. The published construction is exact precisely at power-of-two N and G .

The corrected construction. Cut $\text{FBT}(N)$ at depth $d = \lceil \log_2 G \rceil$ instead of partitioning index ranges. The cut frontier — nodes at depth d together with any leaf shallower than d — roots disjoint complete subtrees covering every leaf; assign those subtrees to the G ranks; each rank evaluates its subtrees by the canonical recurrence of Def. 2 under the enumeration of Def. 3; the cross-rank stage evaluates the internal nodes above the frontier, exactly $2^d - 1$ combines — the $\log_2 G$ -depth all-reduce the published version wanted and did not have. The proof obligation is one sentence: every node of $\text{FBT}(N)$ is still computed exactly once, by the same recurrence and with the same operand binding (Def. 4), so the multiset of FADD operations and the dependency order among them are those of $\text{FBT}(N)$ itself and the root is $R_{\text{canonical}}(N, L)$ for every G . Enumeration discharges it over the same space that refutes the published version: every one of those 44,850 pairs is exact in FP64. The frontier-to-rank assignment does not affect the root — the tree fixes the order — so it is a free load-balancing choice, and the choice matters: under a greedy largest-first deal the worst leaf-count ratio is 1.993 and none exceeds 2, while under the natural contiguous deal it reaches 3.53 and exceeds 2 on 13,446 of them. An export must therefore *specify* the greedy deal.

Prior art, and the residue. TBIK [?] predates this manuscript; it is prior art, not concurrent work, and we claim no priority against it. Its Appendix C, Theorem 1 proves by induction, over the same contiguous-block partition, exactly the theorem our step 1 asserted without proof, under exactly the power-of-two hypotheses that make it true. The summary is uncomfortable and worth stating plainly: the correct part of pinned-FBT is what TBIK had already done, and the part TBIK did not do — arbitrary, non-power-of-two N — is the part

that was wrong. What remains ours is the cut-aligned repair for arbitrary N under canonical heap-index leaf enumeration, together with the properties the export inherits from the ABI and TBIK does not test: invariance across network *topologies* rather than across tensor-parallel sizes inside one node, enforcement below the software stack, and presence-gated fail-stop.

Measured: the repair is invariant, and the checker can fail. We implemented both constructions on PyTorch’s collectives and ran them on a single node with two NVIDIA RTX PRO 6000 Blackwell GPUs over PCIe (no NVLink) and 72 CPU cores with NCCL on the two GPUs and the CPU backend for larger world sizes. Over 32 cases spanning N from 3 to 4,096 (ten of the leaf counts not powers of two — the case TBIK does not cover, and the majority of the sweep), world sizes 2–64 (seven not powers of two), messages from 8 KiB to 128 MiB, and two frontier deals, every one of the 2,414 (case, world size, rank) cells returned a single 64-bit root, equal to the single-process canonical $R_{\text{canonical}}(N, L)$. The result is reportable only because the same checker demonstrably fails: run against the index-range construction as a pre-registered negative control, three of four cases broke as they must — at $N = 1,000$ and $N = 1,500$ it returned three distinct roots across world sizes $\{2, 3, 8\}$ and *none* of its 13 cells equalled $R_{\text{canonical}}$. Where it agrees — ten of 13 cells at $N = 1,024$, all of them at $N = 7$ — it agrees by power-of-two coincidence.

Measured: what it costs, and the answer is unfavourable. The statistic was fixed before the run: geometric-mean overhead against stock `ncclAllReduce` at $G=2$ in FP64 over the swept message sizes, worst case beside it, with a support branch at roughly $1.0\text{--}1.3\times$ and an explicit unfavourable branch. The measurement took the unfavourable branch: over 20 configurations the geometric mean is $2.57\times$ stock, best $1.49\times$, worst $3.91\times$. Two qualifications pull in opposite directions. First, the sweep ran while an unrelated job saturated both GPUs, so the point estimate is *provisional*: substituting the best of the repeats at each configuration for the median gives $2.08\times$, an inflation of about $1.24\times$. Both estimators are far outside the pre-registered support band, so the verdict does not depend on the correction — the point estimate does, and a re-run on a quiet machine is owed. Second, the overhead is not one quantity. At $G=2$ the pinned tree moves $2M$ bytes, one directed send of the full vector plus a broadcast, against a bandwidth-optimal ring’s M per rank: a factor of about two *in bytes* that no engineering removes, though it becomes a time ratio only where stock is bandwidth-bound. The remainder is an export artefact — two dependent library calls issued from Python against one fused NCCL kernel — dominating at small messages ($3.29\times$ at 8 KiB, where the whole stock call takes $57\ \mu\text{s}$) and receding at large ones, where the sweep straddles that factor rather than sitting above it ($1.49\text{--}2.21\times$ between 8 MiB and 128 MiB, the largest measured).

What we claim for pinned-FBT is a correctness property, measured above; what

we do not claim is that it is cheap. It should not be set beside TBIK's 10%: that figure is four H20 GPUs over NVLink on an uncontended node, a different fabric measured a different way. Closing the non-algorithmic part of the gap is an engineering problem. The record above is what it looks like when a construction is believed to discharge the structural obligation and does not.

9 Discussion

What the ABI buys, and where the work went. The contribution is not the compiler's algorithms or the fabric's microarchitecture — both are composed from standard prior art — but the interface observation: if compiler and fabric agree on a size-indexed reduction tree as a shared contract, cross-topology bitwise invariance follows by composition. Every design decision in §§4 and 5 is then either discharging a C-* obligation or establishing an F-* invariant, and none of them changes the shape of the argument. What that buys operationally is that a cluster can be reshaped — $G = 2$ to $G = 4096$, linear to hypercube — without touching user code and still return the same bits: bit-for-bit regression testing across hardware generations, replay debugging that survives migration, and a training run re-executed later on a larger cluster that reproduces its weights. §7.1 measures that today's collectives do not provide this, and §7.2 measures that this pair does, on the instances we could simulate.

When not to use it. Three regimes make the ABI a poor fit, and two of them are consequences of the contract rather than of our implementation. *Interoperability*: the bit result agrees across reshape within one compiler/fabric pair and does not match NCCL, RCCL, ReproBLAS or ExBLAS, whose canonical orders differ. A deployment needing bit-compatibility with an external tool needs a canonical-form translation or an input-permutation adapter that presents leaves in that tool's order — which trades the cross-reshape guarantee for the cross-tool one, and cannot have both. *Strict IEEE 754*: workloads whose reference exercises subnormals, multiple rounding modes or the signed-zero cancellation rule diverge from reference at the affected bits (§3.6); the narrower FTZ+RNE semantics is a scope decision, and a strict-IEEE adder inherits Claim 1 unchanged. *Bandwidth-critical inner loops*: the ABI fixes which operands pair at every level where an unconstrained reducer regroups by locality; §7.4 prices that structurally and §8 the software export empirically and unfavourably. A workload that can tolerate reshape non-reproducibility should use a non-deterministic reducer.

What generalises, and what a real workload would need. Claim 1 requires only a deterministic binary operation and a size-indexed canonical tree. It requires neither floating-point nor associativity, so FP32 and FP16 addition, integer

addition under modular or saturating arithmetic, bitwise OR/AND/XOR and integer MAX/MIN all inherit it by substituting the `alu_op` code — a structural extension, not an empirical generalisation, and we report no measurement for any of them. The extension that matters for a real collective is to tensors, and it is element-wise: reducing a tensor of D elements under an ABI-compliant schedule is D independent scalar reductions sharing one routing configuration and one assignment, so the fabric needs no per-element indexing and the invariant is per element. What that costs is D -fold traffic through the same $\text{FBT}(N)$ schedule rather than the vector pipelining a production collective would use, and our evaluation is at $D = 1$; a tensor-rate implementation is engineering we have not done, and no throughput number here should be read as covering it. Non-reduction collectives — all-gather, scatter, broadcast, alltoall — are orthogonal: they have no internal combine, so their results are bit-identical on any routing discipline. The ABI is specifically the statement that reduction, the one collective with an internal combine, can join them.

10 Related Work

Relative to every axis here, what we claim is the conjunction of §1, not any single conjunct; Table 1 is that statement in one view, and this section treats its rows individually. *Reproducible arithmetic* — ReproBLAS [?], ExBLAS [?], Arteaga et al. [?], and the reproducible-summation algorithms of Ahrens et al. [?] — obtains order-independence by changing the operator (pre-rounding, long accumulators, error-free transformations), so the result is reproducible but is not the FP64 sum any particular order produces, and a wide accumulator is required; compensated summation [?] reduces error without fixing order. We change no operator and add no accumulator, and fix the order instead. *Fixed-order schemes in deployed stacks* pin one axis inside one implementation: oneMKL’s conditional numerical reproducibility is invariant to thread count and input partitioning [?]; RepDL [?] targets bit-level reproducibility of deep-learning training and inference; TBIK [?] is invariant across tensor-parallel size and proves the power-of-two case of the construction §8 repairs, at a measured cost on NVLink far better than our software export’s on PCIe; the “deterministic” modes of NCCL and RCCL hold within one (G , topology) and not across reshape [?,?]. *In-network reduction* is the closest precedent for enforcing order in hardware, and we claim no advantage over it in the speed or cost of the reduction itself. SHARP [?] aggregates inside InfiniBand switches and pins operand order to do so, each aggregation node holding a predefined order of calculation [?]; its specification is equally explicit that reproducibility holds “for a given application-to-system mapping” — per deployment, where our index is N and survives the deployment changing, which a per-mapping order

cannot without standardising switch hierarchies across sites. It also prices the discipline structurally, a non-reproducible mode saving aggregation buffering by $n/2$ in the number of inputs. SwitchML [?] instead carries gradients in 32-bit fixed point through integer-only switch ALUs, so its order-insensitivity is integer associativity rather than a pinned order. Groq’s TSP [?] shows a compiler/fabric determinism contract shipping, but the determinism is of the schedule: no bitwise claim is made, and its all-reduce is keyed to the packaging hierarchy, so reduction order is again a function of how many chips sit in a node. *Hardware-enforced determinism* has a longer line in shared-memory multiprocessing — DMP [?] makes a multithreaded execution deterministic in hardware — but what is fixed there is the thread interleaving, not the arithmetic order of a collective, and the guarantee is per machine rather than per problem size. None of these states the invariant at a compiler/fabric interface indexed by problem size, enforced below the software that could violate it, with a fail-stop failure mode. *Collective-scheduling compilers* [?, ?, ?, ?] search schedule space for throughput, treating reduction order as free; that freedom is exactly what we remove. Our assignment, mapping, routing and pipeline components are all prior art [?, ?, ?]; the contribution is that any choice among them inherits invariance by composition.

11 Limitations

L0 — One unbuildable configuration. Hardware validation covers 39 of the 40 instances. The exception is `maxp_mod_fat` ($N=8192$, $G=4096$, fat-tree), whose five attempted testbenches all hit the wall-clock cap for 58.9 core-hours — but wall-clock was not the binding constraint, and we correct an earlier account that said it was. The instance instantiates 4,192 routers while the adjacency interface carries a 12-bit router id whose top code is the disconnected sentinel, so every one of its 12,288 adjacency writes touches an id that aliases onto a low one. Replayed through the RTL’s own truncation and range guards, the emitted configuration installs 8,223 links of which none match the intended fat-tree and delivers none of 900 sampled leaf-to-leaf routes, all 900 of which deliver on the untruncated fabric. No budget could have completed it. We have since repaired the truncation and re-elaborated: across three attempts under two `xelab` debug configurations, two ran elaboration to completion and exited zero reporting the snapshot built, and zero wrote a simulation kernel — so there is nothing for the simulator to load. We assert no root cause: a tool that exits zero having written no output is misreporting and we have not established why. The exclusion is a truncation defect in the emitter as shipped and, once repaired, a tool limit at this design point; it is neither a wall-clock budget nor a property of the fabric.

L0a — The denominator. On the 39 instances, 2,844 distinct images were attempted, 2,822 settled and passed, 22 did not settle. All but one of those are watchdog fail-stops — their per-PE counters stop at a last retirement plus exactly 20,000 cycles, far inside the testbench budget — and the remaining one is a simulator wall-clock kill. Non-settling runs are not evidence for Claim 1 and we do not count them as such; they are the campaign’s only observation of Property 1 on unmutated compiler output. Coverage is narrower than the space we advertise: the 180 pairs enumerated per instance are 0.82% of the 21,960 triples, refinement is None in every measured configuration, and the remaining axis is covered by Proposition 1 rather than by measurement.

L0b — Configuration-load bypass, measured. Route-table initialisation in every reported run is by hierarchical assignment at time zero rather than the clocked protocol, which costs $\mathcal{O}(G^2)$ simulated cycles. The substitution is measured rather than argued: the re-execution of §7.2 gives the same verdict, root and cycle count under both paths on 1,326 of 1,326 points (100% bit-exact, no exceptions), the clocked path adding only a deterministic prologue of one clock per route-table entry that completes before `program_start`; and corrupting the route table flips PASS to FAIL on 24 of 24 runs, so that path genuinely exercises routing. The residual is scope, not fidelity: the clocked path is uncharacterised above $G = 64$.

L1 — Scope. `fp64_add` is unpipelined. A stateless adder is what makes the arithmetic a pure function of its operand pair, which Claim 1 needs, but it is not a realistic synthesis target; we have run no synthesis and report no $F_{\max}$, slack or utilisation. Pipelining would change *when* a COMPUTE releases its scoreboard bit, not which operand pair it reads, so cycle counts would move and the roots in Table 4 would not. Beyond that: no silicon, no measured same-fabric overhead η (§6), no head-to-head against production collectives on production interconnect, one leaf vector per N , one reduction per program (scoreboard bits are monotone within a program), and a suite whose (size $\times$ topology) cells are too thin for cell-level claims. FTZ+RNE is strictly narrower than IEEE 754-2019. C-sw is soft-enforced: a compile-time post-condition, not a fabric check (§4.9). And Claim 1 has no formal proof: the argument is by code inspection and dynamic checking, not mechanised proof.

L1b — No measured same-fabric overhead at non-power-of-two scale. We report no measured η , and on most of the suite none is needed: §6 shows $\eta = 1$ by construction on the 31 power-of-two instances, where the ABI schedule and recursive doubling are the same DAG. The residual is the 8 instances whose N or G is not a power of two. There the isomorphism does not hold, the ABI may cost something we have not priced, and no baseline was built to price it. The one head-to-head we do report is the software export of §8 against stock `ncclAllReduce`; it prices the export at $G=2$ on PCIe, not the fabric, and it is unfavourable.

	Lin.	Ring	Mesh	Torus	Fat-tr.	Hyp.
8–32	1	2	1	1	1	1
33–256	1	1	1	3	2	1
257–1K	2	2	1	3	2	2
1K–4K		1	1	2	3	2
8K				1	1	1

 ≥ 3
 thin (≤ 2): 23 cells
 empty: 4
30 cells, 40 instances – no cell-level claim is derivable (L5)

Figure 19. Instance-suite coverage: the 40 instances by N -bucket and topology, the grid on which every scale-dependent statement in this paper rests. The one thing to take from it is how little of it is populated — 4 of the thirty cells are empty and 23 of the populated ones hold two instances or fewer, so no cell-level recommendation is derivable and none is made (L5). The full table, with row and column totals, is in the supplemental material.

L5 — What the instance suite cannot support. The 40-instance suite covers the $(N, G, \text{topology})$ space unevenly (Fig. 19), and two consequences bind every scale-dependent statement here. N and G are confounded by construction (Spearman $\rho = 0.871$, $p = 5.6 \times 10^{-13}$), so no result we report separates the influence of problem size from that of worker count. Of the thirty (size $\times$ topology) cells, 4 are empty and 23 hold two instances or fewer, so no cell-level recommendation is derivable and we make none. This is why the rule of §7.6 is reported unconditionally and with held-out intervals: it requires no per-cell association to hold.

L6 — The C-mem margin is thin, and past it the failure is silent. Peak DMEM occupancy over the validated configurations reaches 491 of a hard 512 slots. Claim 1 holds only inside that ceiling and the campaign came within 21 slots of it; past it a run terminates normally with no P_ERROR and returns a wrong root — two *different* wrong roots under two packet-delivery orders (Remark 1). The released emitter refuses such an image; the fabric still cannot detect one.

L10 — No deadlock-freedom property, and no virtual-channel discipline. The channel-dependency graph built from the emitted minimal-distance routing tables is cyclic on every ring instance (cycle length G) and every torus instance (length 6–32), and no escape channel exists: the compiler assigns no virtual channel and the router’s egress buffer is a single FIFO shared across channels, so a second VC would not be an escape even if one were assigned (§4.6). C-route therefore claims destination-correctness and loop-freedom only. Deadlock is prevented by neither compiler nor fabric; it is converted into a recorded fail-stop by the watchdog, which is where most of L0a’s non-settling runs come from.

L11 — Mutation testing covers two classes. We inject address substitution and leaf corruption only; scoreboard-set, forwarding-network and credit-return

mutations are untested, so the figures are a lower bound on the harness’s sensitivity rather than a characterisation of it.

L13 — Two defect classes this evaluation cannot see. This one bounds the evaluation *method* rather than the artifact, and it is the item we would most want a reader building a similar campaign to take away. First, the campaign cannot exercise the FADD’s effective-subtraction path at all: Eq. (5) is strictly positive at every index, so both operands of every FADD in all 2,822 runs carry the same sign. The defect that lived on that path could not have appeared in any number reported here — not at this campaign size and not at any other, because the blindness does not lie along the axis the campaign enumerates: all 7,200 compilations share one leaf generator. Second, and closer to the bone, the oracle against which every root is compared is not an independent model of IEEE-754 addition. It is a transcription of the same adder and it reproduced the same defect bit for bit; on the as-submitted pair the two agree on all 62,320 vectors of the acceptance test while both disagree with correctly-rounded addition on 15,717 of them. A differential test is only as strong as the independence of its two sides, and ours were not independent. Both are cases of an evaluation that cannot detect a class of defect *by construction*, and neither is visible from inside the campaign: no pass rate, coverage figure or additional instance would have exposed either, and both were found by reading the arithmetic against an external standard. Enumerating a parameter space exhaustively says nothing about the axes that space does not span, and the two axes that mattered here — operand sign, and the independence of the oracle — were not parameters of the campaign at all.

12 Conclusion

Cluster-reshape non-determinism in floating-point reductions is an *interface* problem, and the place to fix it is the contract between a collective-scheduling compiler and its fabric. Reduction order, treated as a size-indexed ABI over a canonical full binary tree, is orthogonal to G , topology, assignment, mapping, refinement and schedule, and yields cross-topology bitwise invariance: the 64-bit root is a pure function of N and the leaf vector. What we claim is a conjunction, not a first — that invariant, enforced at the compiler/fabric interface with a fail-stop failure mode, across network topologies rather than resource counts alone, over an unmodified IEEE-754 FP64 operator, for arbitrary non-power-of-two N .

On the 39 hardware-validated members of a 40-instance suite spanning $N \in [8, 8192]$, $G \in [2, 4096]$ and six topologies, the campaign RTL-validated every distinct image the refinement-free face of the configuration space produces:

2,822 of 2,844 settled, every one with a root bit-identical to the canonical one, and 22 did not settle, all but one a watchdog fail-stop. Injected faults establish that the zero-divergence result is non-vacuous on 61–65 of 107 verdicts, the remainder being the halt outcome Property 1 predicts. We price the ABI by construction where the ABI fixes the schedule and by measurement where it does not; the residual — a same-fabric baseline at non-power-of-two scale — is the principal open item in the evaluation.

Two negative results travel with the positive one. The compiler search this campaign was built around is not a ranking problem: the minimising configuration is a tie set on 38 of 39 instances, and what replaces the ranking — a fixed portfolio of 4 assignment algorithms reaching the measured optimum on 38 of 39 held-out instances, transferring to an entirely unseen topology at mean regret 1.0391 [1.0025, 1.0885] and beating every regime map we could fit — is both cheaper and better supported than the distribution it replaces. And two defect classes in our own artifact are invisible to this evaluation by construction: we found them by reading the arithmetic, not by running more configurations. An evaluation that enumerates a space exhaustively can still be blind along an axis that space does not span, and that blindness is not observable from inside the campaign.

Finally, *pinned-FBT* exports the discipline onto existing collective libraries. The index-range construction is exact only at power-of-two N and G ; the cut-aligned repair given here is bitwise reshape-invariant on all 2,414 cells swept, against a pre-registered negative control that broke on three of four cases. Its cost is unfavourable and the statistic was fixed before the run: a geometric mean of $2.57\times$ stock `ncclAllReduce` at $G=2$ in FP64, provisional under GPU contention, of which a factor of about two is the pinned tree’s algorithmic cost at two ranks, in bytes moved. Reshape reproducibility is available in software where the collective is not the bottleneck, and is the wrong purchase inside a bandwidth-critical loop. Fabric-level and software-level enforcement discharge the same structural obligation at very different prices; what makes either one checkable is that the reduction order is enforced rather than intended.

Supplemental material. Three appendices accompany this paper as supplemental material and are referenced above where each is used: the fifteen assumptions stated in full, with the four that are hypotheses of Property 1 alone marked; the catalogue of the eighteen assignment algorithms with champion-class membership and their provenance in the literature, beside the instance-suite coverage table; and the reproducibility record, which lists the released artefacts, states exactly what each per-run verdict is, and records that the software oracle is a transcription of the RTL rather than an independent model of IEEE-754 addition.

References

- [1] R. Rabenseifner, “Optimization of collective reduction operations,” in *ICCS 2004*, vol. 3036 of *LNCS*, pp. 1–9, Springer, 2004.
- [2] NVIDIA, “NCCL: Optimized primitives for collective multi-gpu communication.” <https://github.com/NVIDIA/nvcc1>, 2018–2025.
- [3] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2nd ed., 2002.
- [4] S. Shanmugavelu, M. Taillefumier, C. Culver, O. Hernandez, M. Coletti, and A. Sedova, “Impacts of floating-point non-associativity on reproducibility for HPC and deep learning applications.” arXiv:2408.05148, 2024.
- [5] S. Li, Y. Zhao, R. Varma, O. Salpekar, P. Noordhuis, T. Li, A. Paszke, J. Smith, B. Vaughan, P. Damania, and S. Chintala, “PyTorch distributed: Experiences on accelerating data parallel training,” in *VLDB*, 2020.
- [6] IEEE, “IEEE standard for floating-point arithmetic.” IEEE Std 754-2019 (Revision of IEEE 754-2008), 2019. doi: 10.1109/IEEESTD.2019.8766229.
- [7] H. Topcuoglu, S. Hariri, and M.-Y. Wu, “Performance-effective and low-complexity task scheduling for heterogeneous computing,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 13, no. 3, pp. 260–274, 2002.
- [8] J. A. Lukes, “Efficient algorithm for the partitioning of trees,” *IBM Journal of Research and Development*, vol. 18, no. 3, pp. 217–224, 1974.
- [9] W. J. Dally and B. P. Towles, *Principles and Practices of Interconnection Networks*. San Francisco, CA: Morgan Kaufmann, 2004.
- [10] P. Patarasuk and X. Yuan, “Bandwidth optimal all-reduce algorithms for clusters of workstations,” *Journal of Parallel and Distributed Computing*, vol. 69, no. 2, pp. 117–124, 2009.
- [11] P. Sanders, J. Speck, and J. L. Träff, “Two-tree algorithms for full bandwidth broadcast, reduction, and scan,” in *Parallel Computing*, vol. 35, pp. 581–594, 2009.
- [12] AMD, “RCCL: ROCm collective communication library.” <https://github.com/ROCmSoftwarePlatform/rccl>, 2019–2025.
- [13] P. Balaji and D. Kimpe, “On the reproducibility of MPI reduction operations,” in *IEEE International Conference on High Performance Computing and Communications (HPCC)*, pp. 407–414, IEEE, 2013.

- [14] J. Demmel and H. D. Nguyen, “Fast reproducible floating-point summation,” in *21st IEEE Symposium on Computer Arithmetic (ARITH)*, pp. 163–172, IEEE, 2013.
- [15] R. Iakymchuk, S. Collange, D. Defour, and S. Graillat, “ExBLAS: Reproducible and accurate BLAS library,” *Numerical Reproducibility at Exascale (NRE) Workshop, SC*, 2015.
- [16] S. Collange, D. Defour, S. Graillat, and R. Iakymchuk, “Numerical reproducibility for the parallel reduction on multi- and many-core architectures,” *Parallel Computing*, vol. 49, pp. 83–97, 2015.
- [17] W. Kahan, “Pracniques: Further remarks on reducing truncation errors,” in *Communications of the ACM*, vol. 8, p. 40, 1965.
- [18] Intel Corporation, “Obtaining numerically reproducible results.” Developer Guide for Intel oneAPI Math Kernel Library for Linux, version 2024.0, document 766690, 2024. Accessed 2026-08-29.
- [19] Z. Zhang, X. Ding, J. Yuan, R. Liu, H. Mao, J. Xing, and Z. Liu, “Deterministic inference across tensor parallel sizes that eliminates training–inference mismatch.” arXiv:2511.17826, 2025.
- [20] R. L. Graham, D. Bureddy, P. Lui, H. Rosenstock, G. Shainer, G. Bloch, D. Goldenberg, M. Dubman, S. Kotchubievsky, V. Koushnir, L. Levi, A. Margolin, T. Ronen, A. Shpiner, O. Wertheim, and E. Zahavi, “Scalable hierarchical aggregation protocol (SHArP): A hardware architecture for efficient data reduction,” in *COMHPC*, pp. 1–10, 2016.
- [21] Mellanox Technologies, “Aggregation protocol.” U.S. Patent 10,284,383 B2, 2019.
- [22] D. Abts *et al.*, “A software-defined tensor streaming multiprocessor for large-scale machine learning,” in *ACM/IEEE International Symposium on Computer Architecture (ISCA)*, pp. 567–580, 2022.
- [23] W. J. Dally and C. L. Seitz, “Deadlock-free message routing in multiprocessor interconnection networks,” *IEEE Transactions on Computers*, vol. C-36, no. 5, pp. 547–553, 1987.
- [24] A. Arteaga, O. Fuhrer, and T. Hoefler, “Designing bit-reproducible portable high-performance applications,” in *IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 1235–1244, IEEE, 2014.
- [25] W. Ahrens, J. Demmel, and H. D. Nguyen, “Algorithms for efficient reproducible floating point summation,” *ACM Transactions on Mathematical Software*, vol. 46, no. 3, pp. 1–49, 2020.
- [26] P. Xie, X. Zhang, and S. Chen, “RepDL: Bit-level reproducible deep learning training and inference.” arXiv:2510.09180, 2025.

- [27] A. Sapio, M. Canini, C.-Y. Ho, J. Nelson, P. Kalnis, C. Kim, A. Krishnamurthy, M. Moshref, D. R. K. Ports, and P. Richtárik, “Scaling distributed machine learning with in-network aggregation,” in *NSDI*, 2021.
- [28] J. Devietti, B. Lucia, L. Ceze, and M. Oskin, “DMP: Deterministic shared memory multiprocessing,” in *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pp. 85–96, ACM, 2009.
- [29] Z. Cai, Z. Liu, S. Maleki, M. Musuvathi, T. Mytkowicz, J. Nelson, and O. Saarikivi, “Synthesizing optimal collective algorithms,” in *PPoPP*, 2021.
- [30] A. Shah, V. Chidambaram, M. Cowan, S. Maleki, M. Musuvathi, T. Mytkowicz, J. Nelson, O. Saarikivi, and R. Singh, “TACCL: Guiding collective algorithm synthesis using communication sketches,” in *NSDI*, 2023.
- [31] R. Thakur, R. Rabenseifner, and W. Gropp, “Optimization of collective communication operations in MPICH,” *International Journal of High Performance Computing Applications*, vol. 19, no. 1, pp. 49–66, 2005.